

Technische Hochschule Ingolstadt
Promotionszentrum Kuenstliche Intelligenz/Informatik
Cluster unbemanntes Fliegen

Expose der Dissertation

**Titel der Dissertation ist
folgendermassen, und zwar geht hier
um extrem relevante und wichtige
Forschritte in Wissunschaft und
Technik**

presented by

Max Klemens

Intelligent autonomous Flight Control

Date of submission: June 11, 2026

Supervisor:
Herr Dr. Loesel

Secondary Supervisor:
Prof. Dr. Christian Seidel

Contents

1	Introduction	2
1.1	Motivation	2
1.2	Research Objectives	4
1.3	Contributions	4
1.4	Thesis Overview	5
2	Related Work	6
2.1	Traditional Model-Based Control for Multi-Unmanned Aerial Vehicle (UAV) Payload Transport	6
2.2	Reinforcement Learning (RL) Approaches for UAV Control	7
2.2.1	Single-Agent RL for UAV Flight Control	7
2.2.2	Sim-to-Real RL for Micro-UAV Control	8
2.2.3	Single UAV RL for Payload Transport and Aerial Manipulation	8
2.3	Simulation Frameworks for Multi-UAV Payload Transport	9
2.4	Multi-Agent Reinforcement Learning (MARL) for Multi-UAV Cooperative Tasks	10
2.4.1	Multi-UAV MARL for Swarm Coordination	11
2.4.2	Multi-UAV Collaborative Transport via MARL	11
3	Background	12
3.1	Reinforcement Learning (RL)	12
3.1.1	Fundamentals	12
3.1.2	Proximal Policy Optimization	15
3.2	Multi-Agent Reinforcement Learning (MARL)	16
3.2.1	Multi-Agent Markov Decision Process	17
3.2.2	Independent Proximal Policy Optimization (IPPO)	17
3.2.3	Multi-Agent Proximal Policy Optimization (MAPPO)	17
3.3	Quadrotor Dynamics	18
3.3.1	Coordinate Frames and State Representation	18
3.3.2	Sensors and State Estimation	18
3.3.3	Quadrotor Dynamics	19
3.3.4	Control Action Parameterizations	20
3.4	Quadrotors with Cable-Suspended Payloads	21

4	Training Multi-Agent Reinforcement Learning Policies for Quadrotors with Payload	23
4.1	Problem Formulation	23
4.2	CrazyMARL	25
4.2.1	Simulation Environment	25
4.2.2	Observation Mapping	26
4.2.3	Action Space	27
4.2.4	Randomization	27
4.2.5	Reward Design	30
4.3	Training	35
4.3.1	Policy Architecture	37
4.3.2	Hyperparameters	37
5	Evaluation and Experiments	38
5.1	Experimental Setup	38
5.2	Single-Quadrotor Payload Control	39
5.3	Multi-Quadrotor Payload Transport	43
5.3.1	Two Quadrotors	43
5.3.2	Three Quadrotors	46
5.3.3	Six Quadrotors	49
5.4	Comparison with Baseline	50
5.5	Generalization	53
5.6	Sim-to-Real Transfer	54
5.7	Experimental Results Summary	54
6	Discussion	55
6.1	Key Findings	55
6.2	Implications	56
6.3	Limitations	56
6.4	Future Research Directions	57
7	Conclusion	58
	Bibliography	59

List of Figures

3.1	Reinforcement Learning Framework	13
3.2	Quadrotor Frames	18
3.3	Cable Modeling Approaches	21
4.1	Simulation Environment	25
4.2	Initial States	28
4.3	Reward Components	31
4.4	Action Regularization	34
4.5	Training Metrics	36
5.1	Recovery Error Comparison	40
5.2	Single Quadrotor Harsh Recovery Trajectory	41
5.3	Single Quadrotor Figure-Eight Tracking	42
5.4	Two Quadrotor Harsh Recovery Trajectory	44
5.5	Two Quadrotor Figure-Eight Tracking	45
5.6	Three Quadrotor Harsh Recovery Trajectory	47
5.7	Three Quadrotor Figure-Eight Tracking	48
5.8	Baseline vs RL Performance	50
5.9	Baseline vs Learned Two Quadrotor Recovery	52
5.10	Policy Generalization	53

Abbreviations

API Application Programming Interface

CTBR Collective Thrust and Body Rates

CTDE Centralized Training with Decentralized Execution

Dec-POMDP Decentralized Partially Observable Markov Decision Process

EKF Extended Kalman Filter

GAE Generalized Advantage Estimation

IMU Inertial Measurement Unit

IPPO Independent Proximal Policy Optimization

LV Linear Velocity and Yaw Rate

MAPPO Multi-Agent Proximal Policy Optimization

MARL Multi-Agent Reinforcement Learning

MDP Markov Decision Process

MoCap Motion Capture

MPC Model Predictive Control

PPO Proximal Policy Optimization

PWM Pulse Width Modulation

RL Reinforcement Learning

SGD Stochastic Gradient Descent

SRT Single-Rotor Thrust

UAV Unmanned Aerial Vehicle

1. Introduction

In recent years Unmanned Aerial Vehicles (UAVs) have transitioned from niche research platforms to versatile tools in a wide range of civil and industrial domains. Their ability to navigate complex environments, carry diverse payloads, and coordinate as autonomous teams has unlocked new possibilities. The applications range from rapid disaster response to precision agriculture. Among these, collaborative UAV payload transport with cables is particularly compelling, as it enables the cooperative handling of heavier or irregularly shaped loads, enhances control precision in constrained environments, and improves redundancy and safety, which are key for tasks such as aerial construction, infrastructure inspection, and emergency logistics. However, as applications become more demanding and dynamic, conventional control methods reveal significant limitations in terms of robustness and scalability. In particular, scenarios involving multiple UAVs collaboratively transporting a shared cable suspended load pose challenges that are difficult to address with traditional model-based control approaches. This thesis presents a Reinforcement Learning (RL) framework for decentralized multi-UAV cable-suspended payload transport, aiming to learn policies that stabilize the payload, track the target, and adapt to disturbances in real-time.

1.1 Motivation

UAVs have seen rapid adoption across a variety of civil and industrial applications, including construction site surveying, debris cleanup after natural disasters, search and rescue operations in hazardous environments, agriculture, package delivery, and inspection or maintenance tasks at nuclear facilities [1, 2, 3, 4]. In these contexts, reliable and adaptive control of single and multiple UAVs is critical to ensure mission success and safety.

Traditional model-based control methods for UAVs rely on highly accurate mathematical descriptions of both vehicle dynamics and the task environment. They are often too sensitive to modeling errors and external disturbances such as wind gusts, changes in payload mass, or collisions, which undermines their robustness in real-world operations. Coordinating these controllers for cooperative tasks in UAVs swarms further increases complexity. Centralized schemes can coordinate on a global scale but face limits in scalability, suffer from communication bottlenecks, and single points of failure. Those drawbacks restrict their applicability in the field, especially when the global state is unknown and computing resources are scarce. Decentralized approaches, by contrast, can tolerate disturbances to individual UAV agents or interruptions in communication. Yet, they often cannot guarantee overall performance and may become trapped in local optima [5, 6].

A particularly challenging application for UAVs is aerial payload transport. Cable-suspended payload transport offers several advantages over other attachment methods. By suspending a load beneath one or more UAVs using flexible cables, the system can adapt to varying payload shapes and sizes without the need for specialized fixtures. Cables are lightweight and easily stowed, which decreases the overall system mass and improves flight endurance. Additionally, cable-suspended configurations provide natural compliance that can absorb shocks and reduce the transmission of vibrations from the UAV rotors to the payload. These benefits make cable-based transport particularly attractive for missions requiring the delivery of sensitive equipment. However, the suspended load can exhibit significant pendulum-like oscillations, which complicate stabilization and trajectory tracking. When multiple UAVs cooperate to carry a shared suspended payload, coordination becomes even more complex, as each vehicle’s motion affects the tension distribution and dynamic behavior of the cables. Traditional control approaches often rely on simplified models, which break down during aggressive maneuvers or in response to external disturbances (e.g., gusts of wind, abrupt payload shifts, collisions, landing, and takeoff). Moreover, designing decentralized controllers that guarantee safe load distribution and robust performance across all UAVs remains an open research problem [7].

RL offers a promising alternative by leveraging data-driven policy optimization to adapt to uncertainty and complex dynamics automatically. RL agents can learn flexible control strategies directly from interaction with a simulator or the real world, showing increased resilience to disturbances and unmodeled effects. Recent studies have demonstrated the potential of both single-UAV and multi-UAV RL in tasks such as target tracking, formation flight, achieving performance on par or exceeding classical controllers under challenging conditions [8, 9, 10, 11, 12].

The goal of this thesis is to develop a deep RL approach for coordinated multi UAV payload transport, with a focus on cable suspended loads. By designing an efficient training pipeline that supports decentralized learning, we aim to achieve control policies that can stabilize the suspended payload, distribute load forces evenly among cooperating UAVs, and adapt to disturbances in real time, while tracking the payload position. Such policies promise to be more robust, safer, and more autonomous than existing methods, enabling scalable multi UAV teams to perform complex transport missions in unstructured and dynamic environments.

We implement our approach on the popular Crazyflie 2.1 research platform, which is commonly used in previous work on UAV control and cooperative transport [11, 12, 13, 14].

1.2 Research Objectives

To address the challenges outlined above, this work is guided by the following research objectives:

- **Formulate the decentralized multi-UAV payload transport problem:** Define a suitable multi-agent Markov Decision Process (MDP) for cable-suspended payload transport, specifying local observation and action spaces for each UAV, and determine how to coordinate through decentralized policies.
- **Design and implement an efficient training pipeline:** Develop a high-performance simulation framework for multiple quadrotors carrying a payload to enable scalable policy learning.
- **Investigate reward design:** Analyze how to decompose the overall objective into separate tracking and stability components, and study the effect of different components in the reward function.
- **Evaluate robustness under disturbances:** Test the learned policies in simulation scenarios, especially under harsh conditions with disturbances, to assess resilience compared to classical controllers.
- **Demonstrate scalability to multiple agents:** Show that the proposed decentralized approach can handle varying numbers of UAVs, maintaining performance as the team size increases.

1.3 Contributions

This thesis makes the following key contributions:

- **Decentralized Multi-Agent Reinforcement Learning (MARL) for Cable-Suspended Payloads:** We formulate the multi-UAV payload transport problem as a decentralized MARL task, enabling each UAV to act based on local observations while still achieving coordinated global behavior under harsh dynamic conditions.
- **High-Performance GPU-Parallel Training Pipeline:** We develop a GPU-parallelized framework in JAX and MJX that simulates cable-suspended payload transport, capturing the complex dynamic interactions among multiple UAVs. This pipeline is optimized for contact-rich scenarios, and future work could leverage the contact dynamics to learn aerial manipulation and other complex tasks.
- **Modular Reward Design:** We propose a modular reward design that separates tracking, stability, and safety objectives, allowing for excellent tracking performance while maintaining stability and robustness.

1.4 Thesis Overview

This thesis is structured as follows. In Chapter 2, we review traditional model-based controllers for multi-UAVs payload transport and survey existing RL approaches for UAV control, including sim-to-real methods and cooperative transport via MARL. Chapter 3 introduces the necessary background on RL foundations, multi-agent Markov decision processes, policy optimization with Proximal Policy Optimization (PPO), and quadrotor dynamics including cable-suspended payload models. In Chapter 4, we present our CrazyMARL framework, detailing the Decentralized Partially Observable Markov Decision Process (Dec-POMDP) formulation, observation and action spaces, and the high-performance JAX/MuJoCo-based simulation and training pipeline. Chapter 5 describes the evaluation methodology and experimental results, demonstrating policy performance in simulation on single- and multi-quadrotor payload transport tasks under harsh conditions and trajectory tracking scenarios. Chapter 6 discusses the lessons learned, practical implications, limitations of our approach, and outlines directions for future work. Finally, Chapter 7 concludes by summarizing the main contributions and highlighting potential extensions for real-world deployment of multi-UAV payload transport policies.

2. Related Work

This chapter reviews prior work on the control of multirotor UAVs transporting cable-suspended payloads. We organize the discussion by first examining traditional model-based control strategies, which laid the foundation for understanding the dynamics and cooperative planning. Building upon these classical approaches, we then highlight recent advances in RL for single-UAV control and payload transport, before considering simulation frameworks that enable high-throughput algorithm development. Finally, we turn to MARL [15], which offers decentralized learning solutions for multi-UAV swarm coordination and collaborative payload transport. A recent survey by [7] provides a comprehensive overview of various methods for aerial cable-suspended transport with quadrotors, which we build upon here.

2.1 Traditional Model - Based Control for Multi - UAV Payload Transport

Early work on cooperative payload transport relied on physics-based models and carefully designed feedback controllers. These approaches provided essential insights into how multiple UAVs interact with a shared cable-suspended payload, forming the baseline for later RL-based and MARL methods.

Early work formulated the coupled dynamics of multiple quadrotors and their suspended payload, enabling coordinated planning and stabilization beyond quasi-static assumptions [16]. Similarly, [17] proposed a virtual-structure formation with adaptive dynamic compensators, achieving robust rod-shaped payload positioning under modeling errors and wind disturbances, critically, without requiring inter-UAV communication. These works demonstrate that carefully derived models and controllers can ensure stable transport, but they also highlight the complexity of explicitly modeling every disturbance and interaction.

Building on these foundational models, [18] derived full rigid-body dynamics on nonlinear configuration spaces and designed controllers that guarantee payload pose tracking with provable stability. In parallel, [19] showed that internal cable tension enables decentralized stabilization of any payload orientation, whereas zero-tension schemes render the payload's attitude uncontrollable. To handle parameter uncertainties, [20] shaped each drone's apparent dynamics, demonstrating stable collaborative transport even under worst-case conditions. These contributions collectively illustrate that traditional controllers can achieve robust performance, but they often require precise knowledge of vehicle and payload parameters.

More recent work has pushed toward online optimization techniques. [21] developed a Model Predictive Control (MPC) framework that plans full six degree of freedom payload trajectories while respecting thrust, collision, and cable tension limits for teams of up to ten UAVs. In a similar direction, [22] optimized payload trajectories and inter-drone spacing within an MPC framework, validating performance on both simulation and experiments. To address real-time computational demands, [23] introduced a distributed trajectory optimization approach, parallelizing computation across agents to enable real-time replanning in cluttered environments. These MPC-based methods demonstrate how traditional control can be scaled to larger teams, but they still rely on accurate models and often incur significant computational overhead.

Analyses of equilibrium and stability help inform both controller design and the motivation for learning-based methods. For example, [24] analyzed equilibrium conditions and stability for two-UAV beam transport without communication, showing that internal forces induced by cable angles enhance attitude robustness under uncertain parameters. [13] integrated sampling-based kinodynamic planning with a dynamics-aware optimizer to navigate UAV teams through confined spaces, achieving lower tracking errors and reduced energy consumption compared to payload-only planners. Most recently, [25] improved the agility and robustness of UAV teams carrying cable-suspended payloads, while avoiding dependence on explicit state estimation of the payload. These studies underscore both the progress and limitations of purely model-based approaches, motivating the exploration of learning-based techniques that can adapt online to uncertainties.

2.2 RL Approaches for UAV Control

While traditional model-based controllers offer provable guarantees, they can struggle with highly nonlinear dynamics, unmodeled disturbances, and complex payload interactions. RL (RL) methods are gaining traction for UAV control because they can adapt to complex and uncertain dynamics while reducing the need for explicit modeling. In the context of payload transport, RL offers the potential to learn control policies that implicitly account for cable dynamics and multi-agent coupling, paving the way toward more flexible and robust systems. We first review general single-UAV RL methods and then focus on sim-to-real RL strategies for micro-UAV platforms.

2.2.1 Single-Agent RL for UAV Flight Control

Initial applications of RL to UAV control focused on single-agent scenarios to validate that learning-based controllers can match or exceed traditional PID and model-based methods. For example, [26] evaluated Deep Deterministic Policy Gradient (DDPG) [27], Trust Region Policy Optimization (TRPO) [28], and PPO [29] for inner-loop attitude control in a quadrotor simulation. Their results demonstrated that RL-based controllers can achieve stability and responsiveness on par with or better than classical PID controllers. Building on this insight, [8] proposed a neural-network-based controller trained via RL to stabilize a quadrotor from harsh initial conditions, like being thrown upside-down. The resulting policy achieved accurate step-response behavior and robust real-world stabilization in both simulation and hardware experiments, showing that RL can handle aggressive maneuvers.

In highly dynamic scenarios at the edge of quadrotor agility, [10] compared optimal control and RL in autonomous drone racing. They found that a RL-based policy could execute aggressive maneuvers near the vehicle’s limits and achieve competitive lap times, illustrating that learned policies can capture complex dynamics more effectively than hand-tuned controllers. More recently, [30] developed a multitask RL framework enabling a single quadrotor to execute high-speed stabilization, velocity tracking, and autonomous racing tasks using a unified policy, illustrating the versatility and efficiency of learned controllers in complex scenarios.

These single-agent studies set the stage for extending RL methods to payload transport, where the coupling between vehicle and payload introduces additional challenges.

2.2.2 Sim-to-Real RL for Micro-UAV Control

Sim-to-real transfer remains a challenge in RL for UAV platforms. In [31] it is demonstrated that domain randomization over physical parameters enables a single policy trained in simulation to generalize zero-shot to quadrotors of varying sizes. Despite these advances, sim-to-real discrepancies under real-world conditions still pose significant challenges in practice, especially for micro-UAVs like the Crazyflie 2.1, which have low inertia and constrained onboard computation. These limitations have led to the development of specialized RL strategies designed to bridge the sim-to-real gap on micro-UAVs.

Abstracting actions as Collective Thrust and Body Rates (CTBR) transfers better than low-level rotor Pulse Width Modulation (PWM) or thrust commands [32]. Leveraging such insights, [12] used an asymmetric actor-critic framework, where the training process has access to privileged simulator information, while the deployed policy uses only realistic observations. This approach yielded zero-shot transfer to Crazyflie hardware within seconds of simulated learning. [14] also managed to transfer by combining careful reward shaping and curriculum schedules, stabilizing training, and ensuring safe initial flights on the real quadrotor.

Table 2.1 summarizes these and other recent RL-based approaches for Crazyflie control. The table highlights how observation spaces, action representations, and sim-to-real techniques vary across studies, and how they achieved robust flight control under real-world conditions.

2.2.3 Single UAV RL for Payload Transport and Aerial Manipulation

Extending single-UAV RL methods to payload transport, researchers have sought to address the additional complexity introduced by cable-suspended loads. [36] introduced model-based meta-RL that adapts quickly to unknown payload dynamics, updating its model mid-flight to maintain stability. By contrast, [37] combined a Lyapunov-based nonlinear controller with RL, resulting in a hybrid controller that rapidly converges and robustly manages payload disturbances. These single-UAV payload studies pave the way toward multi-UAV RL solutions by demonstrating that RL can implicitly handle the coupling between vehicle and payload dynamics.

2.3 Simulation Frameworks for Multi-UAV Payload Transport

While RL methods promise adaptability, they require large amounts of simulated data for training. Simulation frameworks thus play a central role in developing control algorithms and enabling RL workflows. A comprehensive survey in [38] reviews a wide variety of existing simulators, highlighting how UAV simulators accelerate algorithm development.

Specialized payload-transport simulators, such as RotorTM [39], accurately model cable-suspended dynamics but rely on CPU-based physics, resulting in slower runtimes that hinder sample-intensive RL training. To address this bottleneck, GPU-accelerated frameworks have emerged. The Aerial Gym Simulator, built on NVIDIA Isaac Gym, runs

Table 2.1: Summary of recent RL approaches for Crazyflie control. In the table, $\mathbf{p}$ denotes position, $\mathbf{e}$ the position errors, $\mathbf{v}$ velocity, $\mathbf{R}$ a rotation matrix (orientation), and $\boldsymbol{\omega}$ angular velocity. For details, refer to the original paper. We group PWM and motor thrust commands as Single-Rotor Thrust (SRT).

Paper	Observation $\mathcal{O}$	Action $\mathcal{A}$	Reward r	Sim-to-Real
[31]	$(\mathbf{e}_p, \mathbf{e}_v, \text{vec}(\mathbf{R}), \mathbf{e}_\omega) \in \mathbb{R}^{18}$	SRT	$r_t = -c_t, \quad c_t = (\ \mathbf{e}_p\ ^2 + \alpha_v \ \mathbf{e}_v\ ^2 + \alpha_\omega \ \mathbf{e}_\omega\ ^2 + \alpha_a \ a\ ^2 + \alpha_R \cos^{-1} \frac{\text{tr}(\mathbf{R})-1}{2}) \Delta t$	Domain randomization across most physical parameters Motor delay model & motor noise Sensor noise injection Normalized thrust input Sim-to-sim verification Real-world deployment on multiple platforms
[33]	$(x_t, \mathbf{e}_t, a_{t-1}) \in \mathbb{R}^{20},$ $x_t = (\mathbf{p}_t, \mathbf{v}_t, \mathbf{q}_t, \boldsymbol{\omega}_t) \in \mathbb{R}^{13}$ (history $H=2$)	SRT & CTBR (PWM / AttRate / Att)	$r_t = -(\ \mathbf{e}_t\ ^2 + 10^{-4} \ a_t\ ^2 + 10^{-3} \ a_t - a_{t-1}\ ^2 + 10^{-3} \ \boldsymbol{\omega}_t\ ^2) + r_f,$ $r_f = -100$ if terminated	Domain randomization ($k_F, \Delta t, m, I, T_m, k_m$) Gaussian & uniform sensor noise; OU actuator noise Motor delay model Bayesian optimisation of sim parameters (1 h real flight data)
[14]	$(e_W^{1:10}, \mathbf{v}, \text{vec}(\mathbf{R})) \in \mathbb{R}^{42}$ (e_W^k : rel. pos. to next 10 trajectory ref pts.)	CTBR	$r_t = r_{\text{track}} - \lambda \ a_t - a_{t-1}\ ^2,$ $\lambda = 0.4$ ($r_{\text{track}} \propto -\ e_W\ $) also evaluated other rewards	System identification of m, I, k_f, T_m Selective domain randomization (only sensitive params m, k_f) Motor delay model Smoothness penalty on Δa Large-batch PPO training
[34]	$(\mathbf{e}, \mathbf{q}, [r, p, y], \mathbf{v}, \boldsymbol{\omega}, \mathbf{a}_{t-1})$	SRT (PWM)	$r_t = 2 - 2.5\ \mathbf{e}\ - 1.5\ [r, p]\ - 0.05\ \mathbf{v}\ - 0.05\ \boldsymbol{\omega}\ - 0.1\ \mathbf{a}_t\ $	Residual-dynamics skill discovery with orthogonality constraints Real-data fine-tuning
[12]	$(\mathbf{p}, \mathbf{v}, \text{vec}(\mathbf{R}), \boldsymbol{\omega}, a_{t-1:t-H}) \in \mathbb{R}^{18+4H}$	SRT (RPM)	$r_t = -(w_p \ \mathbf{p}\ ^2 + w_v \ \mathbf{v}\ ^2 + w_R \phi(\mathbf{R}) + w_\omega \ \boldsymbol{\omega}\ ^2 + w_a \ \mathbf{a}_t\ ^2) + c_{\text{alive}}$	Zero-shot transfer without domain randomization Motor delay model Action-history observations Asymmetric actor-critic with privileged critic Force/torque disturbances Gaussian sensor noise
[35]	$(\mathbf{e}_i, \mathbf{v}_i, \text{vec}(\mathbf{R}_i), \boldsymbol{\omega}_i, \{\delta \tilde{\mathbf{p}}_{ij}, \tilde{\mathbf{v}}_{ij}\}_{j=1}^Q)$	SRT	$r_t = r_{\text{pos}} + r_{\text{vel}} + r_{\text{ori}} + r_{\text{spin}} + r_{\text{act}} + r_{\delta \text{act}} + r_{\text{rot}} + r_{\text{yaw}} + r_{\text{collision}}$ $r_t = \alpha_{\text{pos}} \ \delta \mathbf{p}_i\ ^2 + \alpha_{\text{vel}} \ \mathbf{v}_i\ ^2 + \alpha_{\text{ori}} \mathbf{R}_{22} + \alpha_{\text{spin}} \ \boldsymbol{\omega}_i\ ^2 + \alpha_{\text{act}} \ \mathbf{f}_i\ ^2 + \alpha_{\delta \text{act}} \ \mathbf{f}_i(t) - \mathbf{f}_i(t-1)\ ^2 + \alpha_{\text{rot}} \frac{\text{tr}(\mathbf{R}_i)-1}{2} + \alpha_{\text{yaw}} \mathbf{R}_{00} + \alpha_{\text{col}} \mathbf{1}_{\text{collision}}$	Swarm of quadrotors Domain randomization (mass, inertia, thrust, etc.) Motor delay model & OU motor noise Uniform/Gaussian sensor noise injection Simplified collision & downwash models Zero-shot transfer demonstrated on Crazyflie swarms

thousands of multirotor instances in parallel with GPU-based physics and sensor rendering, dramatically increasing throughput [40]. Extending these capabilities, the simulator OmniDrones offers GPU-parallelized collaborative simulations of multirotor systems with rigid-link payloads, enabling multi-agent payload transport scenarios [41]. While OmniDrones cannot yet model flexible cable dynamics and incurs some CPU-GPU synchronization overhead as multi-UAV cable-transport tasks grow in complexity, it represents a critical step toward high-throughput training of MARL policies. These simulation frameworks, by bridging the gap between physics fidelity and computational efficiency, accelerate the development of both single-agent and multi-agent RL methods for payload transport.

2.4 MARL for Multi-UAV Cooperative Tasks

MARL has emerged as a powerful framework for multi-agent cooperative tasks, enabling agents to learn coordinated behaviors in dynamic environments. The key challenge in MARL is addressing non-stationarity. As each agent learns, the environment dynamics from any other agent’s perspective change. To address this, early cooperative MARL approaches leveraged a Centralized Training with Decentralized Execution (CTDE) paradigm to stabilize learning.

Multi-Agent Deep Deterministic Policy Gradient (MADDPG) employs a shared critic during training to stabilize inter-agent learning and coordinate policies [42]. However, relying on centralized critics introduces scalability and communication overhead as the number of agents increases. More recent work has shown that PPO-based MARL methods can achieve comparable coordination with simpler architectures. For instance, Multi-Agent Proximal Policy Optimization (MAPPO) extends PPO by using a centralized value function during training and has demonstrated performance on par with earlier CTDE methods across standard benchmarks [43]. Findings from [44] show that Independent Proximal Policy Optimization (IPPO), where each agent trains its own PPO instance without any shared critic or centralized state, often matches or even outperforms MAPPO on tasks such as the StarCraft Multi-Agent Challenge (SMAC). Benchmarking studies confirm that MAPPO nearly matches or exceeds other MARL algorithms, while maintaining a minimal and straightforward architecture [45].

Real-world MARL implementations have demonstrated the viability of decentralized learning for collaborative robotics tasks similar to multi-UAV payload transport. For example, [46] developed decentralized policies for multi-robot manipulation under uncertainty. [47] demonstrated cooperative payload transport across uneven terrain without retraining, and [48] enabled formation control with obstacle avoidance using only local observations. Other works, such as [49], applied decentralized MARL to coordinate multiple robots towing loads with complex dynamics like quadruped platforms. These results highlight the suitability of PPO-based MARL methods (e.g., IPPO and MAPPO) for multi-UAV payload transport, since they can scale to larger teams and adapt to dynamic interactions without requiring explicit global state estimation during execution.

2.4.1 Multi-UAV MARL for Swarm Coordination

In parallel to payload-specific MARL research, there has been significant progress in multi-UAV swarm coordination, which provides valuable insights for multi-UAV payload transport. [11] introduced attention-based neural networks enabling collision avoidance and navigation in UAV swarms, transferring policies trained with up to 32 agents directly to real hardware. [50] addressed formation flight with obstacles through a two-stage RL pipeline employing attention mechanisms, and validated swarm performance through real-world experiments. These swarm coordination advances, particularly in learning decentralized behaviors based on local observations, suggest strategies for coordinating multiple UAVs carrying a shared payload.

Lightweight perception-driven policies have also emerged. [51] developed a LiDAR-based multi-agent navigation policy trained in dense simulation environments, demonstrating successful real-time collision-free flight on hardware with limited computation capabilities. [52] applied deep RL to collaborative pursuit and evasion scenarios, enabling coordinated multi-UAV target herding and obstacle avoidance. All of these works emphasize the importance of decentralized execution, local perception, and scalable architectures. These are properties that are equally critical when UAV teams carry cable-suspended payloads and must adapt quickly to disturbances and other agents' actions.

2.4.2 Multi-UAV Collaborative Transport via MARL

Cooperative payload transport has been well studied under traditional control frameworks, but MARL-based approaches to this problem remain relatively new. Insights from single-UAV RL methods, such as rapid adaptability and robustness to payload disturbances, naturally motivate extensions to multi-UAV scenarios. In these settings, MARL can facilitate decentralized, cooperative strategies that adjust in real time to the complex, coupled dynamics of cable-suspended loads.

For instance, [53] proposed a centralized RL method for two UAVs jointly carrying a cable-suspended payload, demonstrating stable transport in simulation. Building on the idea of learned coordination, [54] introduced an RL-based trajectory planning approach for multi-UAV load transportation, optimizing the transport path while ensuring both stability and safety under uncertainty. Complementing these algorithmic advances, [41] presented OmniDrones, a GPU-parallelized simulation framework that supports payload configurations via rigid links. Although OmniDrones does not yet model flexible cables and has not reported any real-world hardware transfers, it shows promise for the payload transport domain by enabling high-throughput training of multi-UAV policies.

While these studies collectively establish the feasibility of MARL for multi-UAV payload transport, several challenges remain. Key questions include how to scale learned policies to teams, how to maintain stability under agile movements, more complex payload dynamics, and how to achieve reliable sim-to-real transfer on micro-UAV platforms like the Crazyflie.

In summary, the progression from traditional model-based control, through single-agent RL and high-throughput simulation frameworks, to emerging MARL methods highlights a clear trajectory toward increasingly agile, flexible, and scalable strategies for multi-UAV payload transport.

3. Background

The methods developed in this thesis build on foundational concepts from RL, MARL, and quadrotor dynamics. We first review model-free, on-policy policy-optimization algorithms, introducing MDPs, policy gradients, and PPO. We then extend these ideas to cooperative multi-agent tasks, comparing centralized critics with fully decentralized critics. Next, we present the mathematical model of a quadrotor UAV, covering coordinate frames, state representation, sensor fusion, continuous-time dynamics, and common action-space formulations ranging from velocity commands to direct thrust control. Finally, we explain how the quadrotor state is augmented to include cable-suspended payloads, first via a simple pendulum model and then through multi-link or tendon-based approximations, and how these formulations generalize to cooperative transport by multiple UAVs. These elements are essential for developing a deep RL framework for decentralized multi-UAV cable-suspended payload transport.

3.1 Reinforcement Learning (RL)

Reinforcement Learning (RL) refers to algorithms that learn to make decisions by interacting with an environment through trial-and-error exploration, gradually improving behavior based on received feedback. These methods have been successfully applied to a wide range of sequential decision-making problems, including robotics control, game playing, and resource management [55]. RL algorithms can be broadly categorized into model-free and model-based approaches, as well as on-policy and off-policy methods, each offering different trade-offs between sample efficiency and stability [28].

Model-free methods learn policies or value functions directly from observed interactions without constructing an explicit model of the environment’s dynamics. This simplifies implementation and avoids model bias, but often requires more data to converge compared to model-based approaches.

This work focuses on model-free, on-policy, policy-optimization methods, where the policy used to collect data is the same one being updated. For a comprehensive treatment of RL fundamentals see [56].

3.1.1 Fundamentals

Before introducing the specific algorithm, we first define the key concepts and ideas of RL.

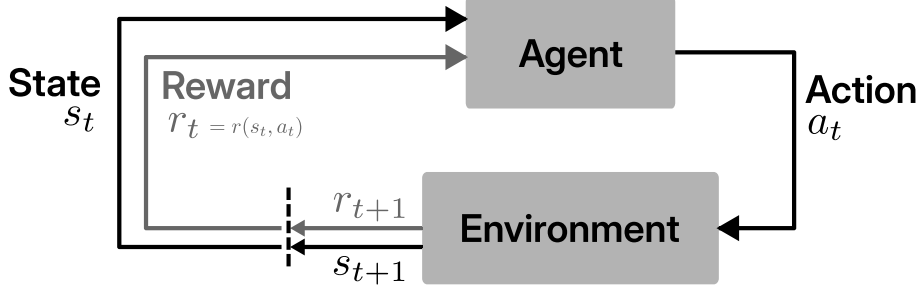


Figure 3.1: RL framework. The agent interacts with the environment by taking actions and receiving rewards, while the environment transitions to new states based on the agent’s actions.

Markov Decision Processes

RL problems are commonly framed as MDPs, defined by the tuple $(\mathcal{S}, \mathcal{A}, P, r, \rho_0, \gamma)$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $P(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ is the transition probability, $r(\mathbf{s}_t, \mathbf{a}_t)$ is the immediate reward, $\gamma \in [0, 1)$ is the discount factor, and ρ_0 is the initial state distribution with $\mathbf{s}_0 \sim \rho_0$. At each time step t , the agent observes $\mathbf{s}_t$, samples and executes $\mathbf{a}_t \sim \pi_\theta(\cdot | \mathbf{s}_t)$, receives $r_t = r(\mathbf{s}_t, \mathbf{a}_t)$, and transitions to $\mathbf{s}_{t+1} \sim P(\cdot | \mathbf{s}_t, \mathbf{a}_t)$. This procedure is visualized in Figure 3.1. In practice, the agent often has access only to a partial observation $\mathbf{o}_t$. We use $\mathbf{o}_t$ or $\mathbf{s}_t$ interchangeably depending on context.

Policies

The agent’s behavior is defined by a policy π_θ , parameterized by $\theta \in \mathbb{R}^d$. A deterministic policy outputs $\mathbf{a}_t = \pi_\theta(\mathbf{s}_t)$, while a stochastic policy samples $\mathbf{a}_t \sim \pi_\theta(\cdot | \mathbf{s}_t)$. We optimize π_θ to maximize the expected return, typically representing it as a neural network mapping states to action distributions.

Reward and Return

The agent’s goal is to maximize the expected return, which is the total discounted reward over time. The reward function $r_t = r(\mathbf{s}_t, \mathbf{a}_t)$ provides immediate feedback to the agent and usually depends on the current state $\mathbf{s}_t$, action taken by the policy $\mathbf{a}_t$ and the next state $\mathbf{s}_{t+1}$. The discounted return from the start is defined as

$$R_0 = \sum_{t=0}^{\infty} \gamma^t r_t, \quad (3.1)$$

where γ is the discount factor that determines the importance of future rewards. The expected return $J(\pi)$ is defined as the expected value of the discounted return

$$J(\pi) = \mathbb{E}[R_0] = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r_t\right]. \quad (3.2)$$

and the goal is to find $\pi^* = \arg \max_{\pi} J(\pi)$. Formally,

$$\begin{aligned} & \text{maximize} \quad \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right] \\ & \text{subject to} \quad \mathbf{s}_0 \sim \rho_0, \quad \mathbf{a}_t \sim \pi(\cdot \mid \mathbf{s}_t), \quad \mathbf{s}_{t+1} \sim P(\cdot \mid \mathbf{s}_t, \mathbf{a}_t). \end{aligned} \quad (3.3)$$

Value and Action-Value Functions

Under a fixed policy π_{θ} , the state-value function $V^{\pi}(\mathbf{s}_t)$ and the action-value function $Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ measure the expected discounted return:

$$V^{\pi}(\mathbf{s}_t) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \mathbf{s}_0 = \mathbf{s}_t \right], \quad Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid \mathbf{s}_0 = \mathbf{s}_t, \mathbf{a}_0 = \mathbf{a}_t \right]. \quad (3.4)$$

Here, $V^{\pi}(\mathbf{s}_t)$ is the expected discounted return starting from state $\mathbf{s}_t$ under π_{θ} , and $Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ is the expected discounted return starting from $\mathbf{s}_t$, taking action $\mathbf{a}_t$, then following π_{θ} . The Bellman equations decompose these functions into immediate reward plus the discounted value of the subsequent state:

$$V^{\pi}(\mathbf{s}_t) = \mathbb{E}_{\mathbf{a}_t \sim \pi(\cdot \mid \mathbf{s}_t)} [r(\mathbf{s}_t, \mathbf{a}_t)] + \gamma \mathbb{E}_{\mathbf{s}_{t+1} \sim P(\cdot \mid \mathbf{s}_t, \mathbf{a}_t)} [V^{\pi}(\mathbf{s}_{t+1})], \quad (3.5)$$

$$Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = \mathbb{E}_{\mathbf{s}_{t+1} \sim P(\cdot \mid \mathbf{s}_t, \mathbf{a}_t)} [r(\mathbf{s}_t, \mathbf{a}_t)] + \gamma \mathbb{E}_{\mathbf{a}_{t+1} \sim \pi(\cdot \mid \mathbf{s}_{t+1})} [Q^{\pi}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1})]. \quad (3.6)$$

The advantage function $A^{\pi}(\mathbf{s}_t, \mathbf{a}_t)$ quantifies the relative value of taking action $\mathbf{a}_t$ in state $\mathbf{s}_t$ compared to the average value of $\mathbf{s}_t$:

$$A^{\pi}(\mathbf{s}_t, \mathbf{a}_t) = Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) - V^{\pi}(\mathbf{s}_t). \quad (3.7)$$

On-Policy Gradient Methods

On-policy methods collect data under the current policy π_{θ} and perform gradient ascent on $J(\theta)$ to ultimately improve the policy towards π^* . The policy gradient can be expressed using the advantage function:

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t \nabla_{\theta} \log \pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t) \hat{A}_t \right]. \quad (3.8)$$

In practice, there are different approaches to approximating the policy gradient $\nabla_{\theta} J(\theta)$. In the next section, we will focus on the PPO algorithm, which is a popular on-policy method, using a surrogate objective function to optimize the policy.

3.1.2 Proximal Policy Optimization

PPO [29] alternates between sampling trajectories using the current policy and performing multiple epochs of stochastic gradient ascent on a clipped surrogate objective. For continuous control, the actor network outputs parameters of a Gaussian distribution,

$$\mathbf{a}_t \sim \mathcal{N}(\mu_\theta(\mathbf{s}_t), \sigma_\theta(\mathbf{s}_t)), \quad (3.9)$$

Here, the actor network (parameterized by θ) maps a state $\mathbf{s}_t$ to a distribution over actions, thereby defining the policy $\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)$. This stochastic sampling is crucial as it enables the agent to explore a continuous range of actions, promoting both exploration and robustness in learning.

PPO builds on the actor-critic framework, where two neural networks, the actor network and the critic network, are optimized simultaneously. The actor network, or policy network, directly outputs the action probabilities or probability distribution. The critic network, often parameterized by ϕ , estimates a value function $V_\phi(\mathbf{s}_t)$ that evaluates the quality of states, or more precisely, the expected return from $\mathbf{s}_t$ under the current policy. The critic's estimate is used to compute an advantage function that quantifies the relative benefit of taking a specific action in a given state. A commonly used method for estimating advantages is Generalized Advantage Estimation (GAE), which effectively reduces variance while introducing a manageable bias that aids learning stability. GAE computes the advantage as

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V_\phi(\mathbf{s}_{t+1}) - V_\phi(\mathbf{s}_t). \quad (3.10)$$

Here, r_t is the reward received at time t , $V_\phi(\mathbf{s}_t)$ is the estimated value of state $\mathbf{s}_t$ from the critic network under the current policy, $\gamma \in [0, 1]$ is the discount factor that weighs future rewards, and $\lambda \in [0, 1]$ is a parameter that adjusts the trade-off between bias and variance in the advantage estimates.

A key contribution of PPO is the use of a clipped surrogate objective designed to restrict the size of policy updates. We can define the probability ratio between the new and the old policies

$$\rho_t(\theta) = \frac{\pi_\theta(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t | \mathbf{s}_t)}, \quad (3.11)$$

Now, the clipped surrogate objective is

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(\rho_t(\theta) \hat{A}_t; \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right], \quad (3.12)$$

where $\hat{A}_t$ is an estimator of the advantage function and ϵ is a hyperparameter defining the clipping range. This objective penalizes overly large deviations from the previous policy, ensuring that updates remain conservative while still allowing for meaningful improvements.

In practice, PPO combines the policy surrogate loss with additional terms, such as a value function loss, which is computed by the critic network, and an entropy bonus, yielding a composite objective:

$$L(\theta, \phi) = \mathbb{E}_t \left[L^{\text{CLIP}}(\theta) - c_1 (V_\phi(\mathbf{s}_t) - V_t^{\text{target}})^2 + c_2 S[\pi_\theta(\cdot | \mathbf{s}_t)] \right], \quad (3.13)$$

where c_1 and c_2 are coefficients that weight the contributions of the value function error and the entropy bonus $S[\pi_\theta](\mathbf{s}_t)$, respectively. The term $(V_\phi(\mathbf{s}_t) - V_t^{\text{target}})^2$ is the critic's squared error against the target value V_t^{target} . This final objective is optimized using stochastic gradient ascent over multiple epochs on the same batch of on-policy samples, updating both actor parameters θ and critic parameters ϕ .

The overall procedure of PPO in an actor-critic setting is summarized in Algorithm 1. In the algorithm, θ denotes the current policy parameters, θ_{old} are the parameters used for generating the on-policy data, U is the number of parallel actors (often parallel simulation environments), T is the number of timesteps per actor rollout (with UT total timesteps per batch), K is the number of epochs over the data, and $M \leq UT$ is the minibatch size. The critic network's parameters ϕ are updated alongside θ whenever the value loss term is backpropagated.

Algorithm 1 Proximal Policy Optimization (Actor-Critic)

```

1: for iteration = 1, 2, ... do
2:   for actor = 1, 2, ..., U do
3:     Run policy  $\pi_{\theta_{\text{old}}}$  (actor network) in the environment for  $T$  timesteps.
4:     Compute advantage estimates  $\hat{A}_1, \dots, \hat{A}_T$  using the critic network's value
       estimates.
5:   end for
6:   Optimize the composite loss  $L(\theta)$  with respect to  $\theta$  (for the actor) and update  $\phi$ 
       (for the critic) using  $K$  epochs and minibatch size  $M \leq UT$ .
7:   Update  $\theta_{\text{old}} \leftarrow \theta$ .
8: end for

```

Overall, PPO strikes a favorable balance between simplicity and performance, making it one of the most widely adopted on-policy algorithms in modern RL applications. The following sections will discuss the multi-agent extensions of PPO that are used in this work.

3.2 Multi-Agent Reinforcement Learning (MARL)

This section provides an overview of the theoretical background of MARL as applied in this work. Our work evaluates both centralized and decentralized training paradigms. In particular, we employ PPO for centralized training, while also exploring decentralized approaches using IPPO and MAPPO, which are detailed in subsequent sections.

3.2.1 Multi-Agent Markov Decision Process

A cooperative multi-agent RL problem can be formalized as a Dec-POMDP[57], defined by the tuple

$$(\mathcal{N}, \mathcal{S}, \{\mathcal{A}^i\}_{i=1}^N, P, r, \{\Omega^i\}_{i=1}^N, O, \rho_0, \gamma) \quad (3.14)$$

where $\mathcal{N} = \{1, \dots, N\}$ is the set of agents; $\mathcal{S}$ is the set of global states (with $\mathbf{s}_0 \sim \rho_0$ denoting the initial state distribution); $\mathcal{A}^i$ is the action space of agent i , and the joint action space is $\mathcal{A} = \prod_{i=1}^N \mathcal{A}^i$; $P(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ is the transition function, where $\mathbf{a}_t = (\mathbf{a}_t^1, \dots, \mathbf{a}_t^N) \in \mathcal{A}$; $r(\mathbf{s}_t, \mathbf{a}_t) \in \mathbb{R}$ is the common team reward received by all agents; Ω^i is the observation space of agent i , and $O(\mathbf{o}^1, \dots, \mathbf{o}^N | \mathbf{s}_t)$ is the joint observation function; and $\gamma \in [0, 1)$ is the discount factor. At each time step t , each agent i receives a private observation $\mathbf{o}_t^i \in \Omega^i$ sampled from $O(\cdot | \mathbf{s}_t)$ and selects an action $\mathbf{a}_t^i \sim \pi^i(\mathbf{a}^i | \tau_t^i)$ conditioned on its action-observation history τ_t^i . The joint policy $\pi(\mathbf{a} | \tau) = \prod_{i=1}^N \pi^i(\mathbf{a}^i | \tau^i)$ induces an expected discounted return

$$J(\pi) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right]. \quad (3.15)$$

3.2.2 Independent Proximal Policy Optimization (IPPO)

There are two primary approaches for extending PPO to multi-agent settings: IPPO, which employs decentralized critics, and MAPPO, which uses a centralized critic. In IPPO [44], each agent i treats other agents as part of its environment and optimizes its own clipped surrogate objective:

$$L_i(\theta_i) = \mathbb{E}_t \left[L_i^{\text{CLIP}}(\theta_i) - c_1 (V_{\phi_i}(\mathbf{o}_t^i) - V_t^{\text{target}, i})^2 + c_2 S[\pi_{\theta_i}](\mathbf{o}_t^i) \right], \quad (3.16)$$

where

$$L_i^{\text{CLIP}}(\theta_i) = \mathbb{E}_t \left[\min(\rho_t^i(\theta_i) \hat{A}_t^i, \text{clip}(\rho_t^i(\theta_i), 1 - \epsilon, 1 + \epsilon) \hat{A}_t^i) \right]. \quad (3.17)$$

Here, $\rho_t^i(\theta_i) = \pi_{\theta_i}(\mathbf{a}_t^i | \mathbf{o}_t^i) / \pi_{\theta_i^{\text{old}}}(\mathbf{a}_t^i | \mathbf{o}_t^i)$, and $\hat{A}_t^i$ is estimated with GAE using agent i 's local critic V_{ϕ_i} . Each agent collects its own on-policy rollouts $\{(\mathbf{o}_t^i, \mathbf{a}_t^i, r_t)\}$ and performs K epochs of minibatch Stochastic Gradient Descent (SGD) exactly as in single agent PPO. While this simplicity enables straightforward scaling to many agents, it does not explicitly address the non-stationarity introduced by concurrent learning: all stability and performance gains must emerge implicitly from PPO conservative updates and entropy regularization.

3.2.3 Multi-Agent Proximal Policy Optimization (MAPPO)

MAPPO [43] adopts a CTDE paradigm. A shared policy $\pi_{\theta}(\mathbf{a}^i | \mathbf{o}^i)$ is used by all agents at execution, while training uses a centralized critic $V_{\phi}(\mathbf{s}_t)$ that has access to the full state or all agents' observations. The joint objective is

$$L(\theta, \phi) = \mathbb{E}_t \left[L^{\text{CLIP}}(\theta) - c_1 (V_{\phi}(\mathbf{s}_t) - V_t^{\text{target}})^2 + c_2 \sum_{i=1}^N S[\pi_{\theta}(\cdot | \mathbf{o}_t^i)] \right], \quad (3.18)$$

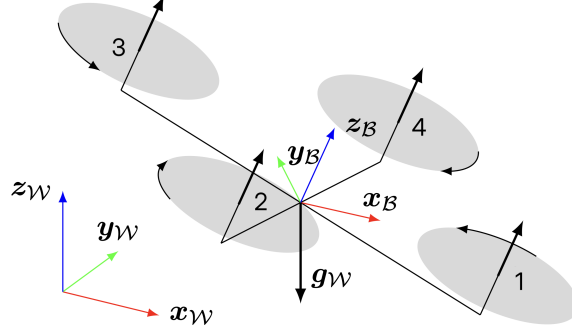


Figure 3.2: Quadrotor frames and propeller numbering. The body-fixed frame $\mathcal{B}$ is centered at the quadrotor's center of mass, with axes aligned to the forward, right, and upward directions. The world frame $\mathcal{W}$ is a fixed global reference frame. Propellers are numbered 1-4 in a clockwise direction when viewed from above. *Adapted from [32].*

where

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(\rho_t(\theta) \hat{A}_t; \text{clip}(\rho_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right], \quad (3.19)$$

and $\rho_t(\theta) = \prod_i \frac{\pi_\theta(\mathbf{a}_t^i | \mathbf{o}_t^i)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t^i | \mathbf{o}_t^i)}$. By conditioning the critic on the true global state, MAPPO obtains lower-variance advantage estimates and improved credit assignment, while each agent remains decentralized at execution, relying only on its local observation $\mathbf{o}^i$. Shared policy parameters promote coordinated behaviors and efficient scaling.

3.3 Quadrotor Dynamics

Quadrotors are rotorcraft UAVs with four fixed-pitch propellers generating upwards thrust. Their mechanical simplicity and agility make them popular for aerial surveillance, package delivery, and search-and-rescue. This section introduces coordinate frames, state representation, onboard sensors, and the continuous-time dynamics and common control action parameterizations for a quadrotor platform.

3.3.1 Coordinate Frames and State Representation

We define the world frame $\mathcal{W}$ and the body-fixed frame $\mathcal{B}$ as shown in Figure 3.2. The origin of $\mathcal{B}$ is at the quadrotor's center of mass, with its x -axis pointing forward, y -axis to the right, and z -axis upward. The quadrotor's position in $\mathcal{W}$ is $\mathbf{p} \in \mathbb{R}^3$, and its linear velocity is $\mathbf{v} = \dot{\mathbf{p}}$. Its attitude is given by $\mathbf{R} \in SO(3)$, mapping vectors from $\mathcal{B}$ to $\mathcal{W}$, and the angular velocity in $\mathcal{B}$ is $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^T \in \mathbb{R}^3$. We collect these into the state

$$\mathbf{x} = (\mathbf{p}, \mathbf{v}, \mathbf{R}, \boldsymbol{\omega}). \quad (3.20)$$

3.3.2 Sensors and State Estimation

The quadrotor's state estimator fuses measurements from a 6-axis Inertial Measurement Unit (IMU) and, when available, an external Motion Capture (MoCap) system. The IMU provides three-axis accelerometer and three-axis gyroscope readings:

$$\mathbf{a}_m = \mathbf{R}^T (\ddot{\mathbf{p}} - \mathbf{g}) + \boldsymbol{\nu}_a, \quad \boldsymbol{\omega}_m = \boldsymbol{\omega} + \boldsymbol{\nu}_\omega, \quad (3.21)$$

where $\mathbf{g} = [0, 0, 9.81]^\top$ and $\boldsymbol{\nu}_a, \boldsymbol{\nu}_\omega$ are noise terms. In motion-capture environments, position and (optionally) attitude measurements are

$$\mathbf{p}_{\text{mocap}} = \mathbf{p} + \boldsymbol{\nu}_p, \quad \mathbf{R}_{\text{mocap}} = \mathbf{R}(\mathbf{I} + \widehat{\boldsymbol{\nu}_R}), \quad (3.22)$$

with $\boldsymbol{\nu}_p, \boldsymbol{\nu}_R$ small errors and $\widehat{\cdot}$ mapping $\mathbb{R}^3$ to $SO(3)$. Stacking these gives the measurement vector

$$\mathbf{y} = [\mathbf{a}_m^\top \quad \boldsymbol{\omega}_m^\top \quad \mathbf{p}_{\text{mocap}}^\top \quad \text{vec}(\mathbf{R}_{\text{mocap}})^\top]^\top = h(\mathbf{x}) + \boldsymbol{\nu}. \quad (3.23)$$

An Extended Kalman Filter (EKF) fuses these measurements to produce $\hat{\mathbf{x}} = (\hat{\mathbf{p}}, \hat{\mathbf{v}}, \hat{\mathbf{R}}, \hat{\boldsymbol{\omega}})$ for use by the low-level controllers.

3.3.3 Quadrotor Dynamics

We augment the state $\mathbf{x} = (\mathbf{p}, \mathbf{v}, \mathbf{R}, \boldsymbol{\omega})$ with propeller speeds $\boldsymbol{\Omega} = [\Omega_1, \Omega_2, \Omega_3, \Omega_4]^\top$, forming

$$\mathbf{x}_{\text{full}} = (\mathbf{p}, \mathbf{v}, \mathbf{R}, \boldsymbol{\omega}, \boldsymbol{\Omega}). \quad (3.24)$$

Let m be the mass of the quadrotor, and let $\mathbf{I} = \text{diag}(J_x, J_y, J_z)$ be its moment-of-inertia matrix expressed in $\mathcal{B}$. Gravity in $\mathcal{W}$ is $g \mathbf{e}_3$ with $\mathbf{e}_3 = [0, 0, 1]^\top$. Each propeller i (numbered 1-4 as in Figure 3.2) produces an upward thrust $f_i \in \mathbb{R}$ along the body-fixed z -axis and a reactive torque $\tau_i \in \mathbb{R}$ about that same axis. We assume the thrust and drag torque of each propeller scale quadratically with the motor speed:

$$f_i(\Omega_i) = c_\ell \Omega_i^2, \quad \tau_i(\Omega_i) = c_d \Omega_i^2, \quad (3.25)$$

where c_ℓ and c_d are the experimentally identified thrust and drag coefficients.

Let $\mathbf{r}_{P,i} \in \mathbb{R}^3$ be the position vector from the body-frame origin to propeller i . We define the total propulsive force in the body frame as

$$\mathbf{f}_{\text{prop}} = \sum_{i=1}^4 f_i \mathbf{e}_3^B, \quad (3.26)$$

and the total torque is

$$\boldsymbol{\tau}_{\text{prop}} = \sum_{i=1}^4 \left(\tau_i \mathbf{e}_3^B + \mathbf{r}_{P,i} \times (f_i \mathbf{e}_3^B) \right). \quad (3.27)$$

Ignoring aerodynamic drag, the continuous-time dynamics are

$$\dot{\mathbf{p}} = \mathbf{v}, \quad \dot{\mathbf{R}} = \mathbf{R} \hat{\boldsymbol{\omega}}, \quad (3.28)$$

$$m \dot{\mathbf{v}} = \mathbf{R} \mathbf{f}_{\text{prop}} - m g \mathbf{e}_3, \quad (3.29)$$

$$\mathbf{I} \dot{\boldsymbol{\omega}} = \boldsymbol{\tau}_{\text{prop}} - \boldsymbol{\omega} \times (\mathbf{I} \boldsymbol{\omega}), \quad (3.30)$$

$$\dot{\boldsymbol{\Omega}} = \frac{1}{k_{\text{mot}}} (\boldsymbol{\Omega}_{\text{cmd}} - \boldsymbol{\Omega}), \quad (3.31)$$

where $\hat{\boldsymbol{\omega}}$ is the skew-symmetric matrix of $\boldsymbol{\omega}$, k_{mot} is the motor time constant, and $\boldsymbol{\Omega}_{\text{cmd}}$ is the commanded motor-speed vector [32].

3.3.4 Control Action Parameterizations

Following [32], we describe three common action-space definitions for quadrotor control:

Linear Velocity and Yaw Rate (LV)

A LV policy outputs a desired body-frame velocity $\mathbf{v}_{\text{des}} = [v_x, v_y, v_z]^\top$ and a yaw rate ω_z :

$$\mathbf{a}_{\text{LV}} = \{v_x, v_y, v_z, \omega_z\}. \quad (3.32)$$

A cascaded low-level controller converts these into collective thrust $c = \|\mathbf{f}_{\text{prop}}\|$ and attitude setpoints, then allocates individual propeller thrusts f_i to achieve the commanded ω_z . LV reduces sample complexity and improves sim-to-real transfer, but cannot leverage full force-torque dynamics for aggressive maneuvers.

Collective Thrust and Body Rates (CTBR)

A CTBR policy outputs total thrust $c \in \mathbb{R}$ and body-rate setpoints $\boldsymbol{\omega}_{\text{des}} = [\omega_x, \omega_y, \omega_z]^\top$:

$$\mathbf{a}_{\text{CTBR}} = \{c, \omega_x, \omega_y, \omega_z\}. \quad (3.33)$$

An inner-loop rate controller computes moments $\mathbf{M} = [M_x, M_y, M_z]^\top$ to track $\boldsymbol{\omega}_{\text{des}}$, and the mixer solves

$$\sum_{i=1}^4 f_i = c, \quad \ell(f_2 - f_4) = M_x, \quad \ell(f_3 - f_1) = M_y, \quad \tau_1 - \tau_2 + \tau_3 - \tau_4 = M_z \quad (3.34)$$

for individual thrusts f_i . CTBR achieves higher agility than LV while remaining more robust than end-to-end approaches.

Single-Rotor Thrust (SRT)

An SRT policy outputs the four individual propeller thrusts:

$$\mathbf{a}_{\text{SRT}} = \{f_1, f_2, f_3, f_4\}, \quad (3.35)$$

where $f_i = c_\ell \Omega_i^2$. This end-to-end formulation grants full authority over $\mathbf{f}_{\text{prop}}$ and $\boldsymbol{\tau}_{\text{prop}}$, enabling the most aggressive maneuvers. Still, it is sample-inefficient and sensitive to modeling errors.

In summary, LV actions abstract away low-level thrust allocation at the cost of dynamic expressiveness, CTBR retains medium-level control over thrust and attitude rates, and SRT provides full end-to-end control at the cost of higher learning complexity and sensitivity to modeling errors.

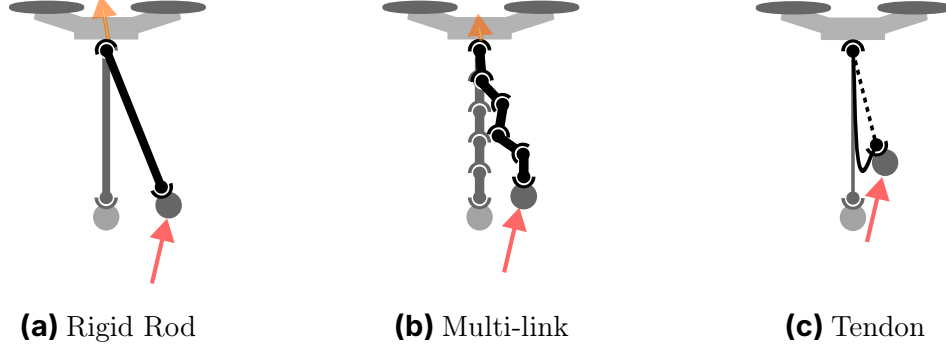


Figure 3.3: Different approaches to modeling a cable-suspended payload. (a) A simple pendulum model with a rigid link and point-mass payload. (b) A multi-link pendulum model with N serial links. (c) A tendon approximation in MuJoCo. We show how each of these models transfers an upward disturbance force to the payload (marked in red) to the quadrotor (resulting force in orange). The taut state is shown translucent behind the disturbed state.

3.4 Quadrotors with Cable - Suspended Payloads

We evaluate multiple approaches to modeling quadrotors with cable-suspended payloads, which are common in aerial manipulation and transport tasks. These models extend the quadrotor state to include cable and payload variables, enabling simulation and control of cooperative transport scenarios. Figure 3.3 illustrates three common modeling approaches: a simple rigid-rod pendulum, a multi-link pendulum, and a tendon approximation in MuJoCo.

A simple rigid-rod pendulum model treats the cable as a massless, inextensible rigid link of length ℓ with a point-mass payload m_L . The payload position $\mathbf{p}_L$ relates to the quadrotor position $\mathbf{p}_Q$ via

$$\mathbf{p}_L = \mathbf{p}_Q - \ell \mathbf{q}, \quad \|\mathbf{q}\| = 1, \quad (3.36)$$

where $\mathbf{q} \in S^2$ is the unit vector along the cable [7]. The augmented state is

$$\mathbf{x}_{\text{aug}} = (\mathbf{p}_Q, \mathbf{v}_Q, \mathbf{R}, \boldsymbol{\omega}, \mathbf{q}, \dot{\mathbf{q}}), \quad (3.37)$$

with $\mathbf{v}_Q = \dot{\mathbf{p}}_Q$ and $\dot{\mathbf{q}}$ capturing payload swing [13].

A multi-link pendulum model represents the cable as N serial, massless links of lengths $\{\ell_i\}$, each with orientation $\mathbf{q}_i \in S^2$. The payload position becomes

$$\mathbf{p}_L = \mathbf{p}_Q - \sum_{i=1}^N \ell_i \mathbf{q}_i, \quad (3.38)$$

and the state is augmented by $\{\mathbf{q}_i, \dot{\mathbf{q}}_i\}_{i=1}^N$. This captures cable sag and higher-order swing modes but increases the state dimension from 8 (rigid link) to $6 + 4N$, where 6 corresponds to $\mathbf{p}_Q, \mathbf{v}_Q, \mathbf{R}, \boldsymbol{\omega}$ [58].

In MuJoCo, cables are approximated as tendon elements. They are massless, inelastic, high-stiffness springs enforcing $\|\mathbf{p}_Q - \mathbf{p}_L\| = \ell$ when taut and acting as slack otherwise. The augmented state is

$$\mathbf{x}_{\text{aug}} = (\mathbf{p}_Q, \mathbf{v}_Q, \mathbf{R}, \boldsymbol{\omega}, \mathbf{p}_L, \mathbf{v}_L), \quad (3.39)$$

where $\mathbf{v}_L = \dot{\mathbf{p}}_L$. Tendon tension $\mathbf{T}$ applies $-\mathbf{T}$ on the quadrotor and $+\mathbf{T}$ on the payload.

For cooperative transport with M quadrotors, each quadrotor i connects to a common payload of mass m_P via its own tendon. The payload dynamics are

$$m_P \ddot{\mathbf{p}}_P = -m_P \mathbf{g} + \sum_{i=1}^M \mathbf{T}_i, \quad (3.40)$$

and each quadrotor's translational dynamics include $-\mathbf{T}_i$ alongside its thrust. The joint state is

$$\mathbf{x}_{\text{aug}} = (\{\mathbf{p}_i, \mathbf{v}_i, \mathbf{R}_i, \boldsymbol{\omega}_i\}_{i=1}^M, \mathbf{p}_P, \mathbf{v}_P), \quad (3.41)$$

capturing multi-agent interactions, payload swing, tension variations, and hybrid dynamics while remaining computationally efficient for simulation and control experiments.

For our work, we model the cable as a tendon. The key distinction of the tendon model is that it only applies forces when the cable is taut and in tension, while the rigid rod and multi-link models forward forces even when the cable is slack, as illustrated in Figure 3.3. This difference is particularly significant in the interaction of the system with the ground and in the interplay between the quadrotors during very dynamic movements.

4. Training Multi-Agent Reinforcement Learning Policies for Quadrotors with Payload

This chapter presents our methodology for training decentralized MARL policies that enable a team of Crazyflie quadrotors to transport a cable-suspended payload cooperatively. We begin by formulating the cooperative transport task as a Dec-POMDP, defining the state, action, and observation structures. We then describe our high-performance simulation and training pipeline, and finally present the modular reward design used to train our policies.

4.1 Problem Formulation

We model a team of Q Crazyflie quadrotors collaboratively carrying a cable-suspended payload as the Dec-POMDP $(\mathcal{Q}, \mathcal{S}, \{\mathcal{A}^i\}_{i=1}^Q, P, r, \{\Omega^i\}_{i=1}^Q, O, \rho_0, \gamma)$, where $\mathcal{Q} = \{1, 2, \dots, Q\}$ is the set of agents. At each discrete timestep t , the global state

$$\mathbf{s}_t = (\{\mathbf{p}_t^i, \mathbf{v}_t^i, \mathbf{R}_t^i, \boldsymbol{\omega}_t^i\}_{i=1}^Q, \mathbf{p}_t^P, \mathbf{v}_t^P) \quad (4.1)$$

encapsulates each quadrotor's position $\mathbf{p}_t^i \in \mathbb{R}^3$, velocity $\mathbf{v}_t^i \in \mathbb{R}^3$, attitude $\mathbf{R}_t^i \in SO(3)$, angular velocity $\boldsymbol{\omega}_t^i \in \mathbb{R}^3$, and the payload's position $\mathbf{p}_t^P \in \mathbb{R}^3$ and velocity $\mathbf{v}_t^P \in \mathbb{R}^3$. The transition model $P(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ corresponds to a single simulation step in MuJoCo MJX, which approximates the continuous quadrotor and payload dynamics (including cables) over the fixed interval Δt .

The initial state is sampled from a distribution ρ_0 . In our case ρ_0 defines harsh initial conditions, such that the policies must learn to recover from them.

Each agent i selects an action

$$\mathbf{a}_t^i = (f_{1,t}^i, f_{2,t}^i, f_{3,t}^i, f_{4,t}^i) \in \mathbb{R}^4 \quad (4.2)$$

where $f_{j,t}^i$ is the thrust of rotor j . The joint action of all agents is $\mathbf{a}_t = (\mathbf{a}_t^1, \dots, \mathbf{a}_t^Q)$.

We define the payload position tracking error as $\mathbf{e}_t^P = \mathbf{p}_t^P - \mathbf{p}_{\text{des},t}^P$, so that $e_t^P = \|\mathbf{e}_t^P\|$ represents the tracking error at time t . The primary objective is to minimize this error while maintaining system stability. This gives us a rough reward structure of:

$$r(\mathbf{s}_t, \mathbf{a}_t) \approx -e_t^P - (\text{stability penalties}) \quad (4.3)$$

We penalize payload tracking error and include additional terms to discourage large payload swing or cable-tension imbalance.

Each agent must observe its own state as well as the payload's state and the position of the other agents to be able to avoid them. Therefore agent i observes

$$\tilde{\mathbf{o}}_t^i = \left(\hat{\mathbf{p}}_t^i, \hat{\mathbf{v}}_t^i, \hat{\mathbf{R}}_t^i, \hat{\boldsymbol{\omega}}_t^i, \mathbf{e}_t^P, \mathbf{v}_t^P, \{\hat{\mathbf{p}}_t^j\}_{j \in \mathcal{Q} \setminus \{i\}} \right). \quad (4.4)$$

Note that $\tilde{\mathbf{o}}_t^i$ is a simplified observation. The exact observation model is defined below.

Here $\hat{\mathbf{p}}_t^i, \hat{\mathbf{v}}_t^i, \hat{\mathbf{R}}_t^i, \hat{\boldsymbol{\omega}}_t^i$ are EKF estimates of its own state and $\mathbf{e}_t^P, \mathbf{v}_t^P$ are the payload state from MoCap. To reduce notational complexity, we will denote the observation component without $\hat{\cdot}$ as $\mathbf{p}_t^i, \mathbf{v}_t^i, \mathbf{R}_t^i, \boldsymbol{\omega}_t^i$. The observation function $O(\mathbf{o}_t^i | \mathbf{s}_t)$ models sensor noise and maps the global state $\mathbf{s}_t$ to each $\mathbf{o}_t^i$. Finally, $\gamma \in [0, 1)$ is the discount factor.

Agents share a single policy conditioned on their current observation $\mathbf{o}_t^i$ and the previous action $\mathbf{a}_{t-1}^i$ and parameterized by θ . The actions are sampled as

$$\mathbf{a}_t^i \sim \pi_\theta(\mathbf{a}_t^i | \mathbf{o}_t^i, \mathbf{a}_{t-1}^i) \quad \text{for } i = 1, \dots, Q. \quad (4.5)$$

For multiple agent scenarios, we sample the actions independently for each agent, so that the joint action is given by $\mathbf{a}_t = (\mathbf{a}_t^1, \dots, \mathbf{a}_t^Q)$.

We seek the shared parameters θ that maximize the expected discounted return

$$\begin{aligned} \text{maximize} \quad & J(\theta) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right], \\ \text{subject to} \quad & \mathbf{s}_0 \sim \rho_0, \\ & \mathbf{s}_{t+1} \sim P(\cdot | \mathbf{s}_t, \mathbf{a}_t), \\ & \mathbf{o}_t^i \sim O(\cdot | \mathbf{s}_t), \\ & \mathbf{a}_t^i \sim \pi_\theta(\cdot | \mathbf{o}_t^i, \mathbf{a}_{t-1}^i) \quad \forall i. \end{aligned} \quad (4.6)$$

By maximizing $J(\theta)$, the agents learn to jointly minimize payload tracking error e_t^P while maintaining stability throughout the cooperative transport task.

We have now formalized the problem as a Dec-POMDP, which allows us to apply decentralized MARL algorithms to learn policies that can handle the complexities of multi-agent coordination in cable-suspended payload transport tasks. The following section describes the CrazyMARL framework, which provides the necessary simulation and training infrastructure to implement this efficiently.

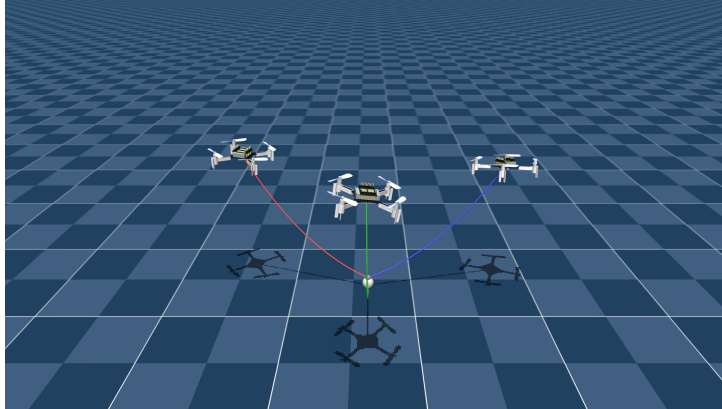


Figure 4.1: Rendering of the MuJoCo simulation with three quadrotors carrying a cable-suspended payload.

4.2 CrazyMARL

We present **CrazyMARL**, an end-to-end JAX-based pipeline for training MARL policies on teams of Crazyflie quadrotors. Our extendable framework implements both learning for a single quadrotor without payload, as well as single-vehicle and cooperative multi-agent cable-suspended payload transport tasks. At its core, the simulation leverages the high-performance MJX backend of the MuJoCo physics engine [59], interfaced through the Brax library for highly parallelized training. Our environments and algorithms are based on JaxMARL introduced by [60]. We expose the simulator as a `JAXMARL.Mabrax` environment, providing a drop-in API for JAX-based RL algorithms and other research algorithms implementing the JaxMARL API.

4.2.1 Simulation Environment

Our framework provides a configurable environment generator for simulating multi-agent cable-suspended payload transport tasks, as well as swarm control of single quadrotors. The environment is designed to be modular and extensible, allowing for easy definition of new scenarios. The environment can be initialized with a configuration that defines the number of quadrotors, the payload mass, cable length, and other relevant parameters. The environment supports both single-agent and multi-agent scenarios, where each agent is a quadrotor with its own observations and actions.

Figure 4.1 shows a rendering of the simulation environment with three quadrotors carrying a cable-suspended payload. The environment is built upon the MuJoCo physics engine, which provides accurate and efficient simulation of rigid body dynamics, including contact and friction. The MJX solver is used to compute the physics updates in parallel on the GPU, allowing for high-performance training of RL policies with up to 32k parallel environments. We can also visualize the simulation using MuJoCo’s rendering capabilities, which provide a realistic representation of the quadrotors, payload, and cables.

The quadrotors are modeled as rigid bodies with four rotors, each producing thrust and a small torque. We optionally reduce the quadrotor’s collision mesh to a rectangular prism to speed up collision detection. The payload is modeled as a rigid body sphere suspended by cables, which are represented as tendons in MuJoCo. The shape and attachment points

of the payload are configurable. The cables connect the quadrotors to the payload and can be configured with different lengths and properties. The environment also supports obstacles, which have not yet been further utilized in our work, but could be a good basis for future work on obstacle avoidance or dynamic interactions with the environment.

We implement the JaxMARL API for multi-agent environments based on Gymnax [61]. This wraps our environment in a standardized interface that can be used with any compatible RL algorithm. The wrapper adds an abstraction layer to the simulation. At each timestep, we call a step function with a set of actions for each quadrotor, and the environment returns the updated state, which is then used to compute the rewards and observations for each agent.

4.2.2 Observation Mapping

In practice, we slightly adjust the observation structure defined in Equation (4.4). Each agent i still receives a local observation vector $\mathbf{o}_t^i$ that contains its own state, the payload's state, and the relative positions of other agents. This ensures that each agent can act based on its measurements and the shared payload information, while not directly observing other agents' internal states. To define the observation space, we first define the global observation vector $\mathbf{o}_t$ that contains all relevant information about the system state at time t . This also serves as the basis for computing rewards and termination conditions and allows for a centralized training setup.

Global Observations

At each control step t , the simulation environment produces a high-dimensional state vector. We map this to a global observation vector

$$\mathbf{o}_t = \left[\underbrace{\mathbf{e}_t^P}_3, \underbrace{\mathbf{v}_t^P}_3, \underbrace{\{\mathbf{r}_t^i, \text{vec}(\mathbf{R}_t^i), \mathbf{v}_t^i, \boldsymbol{\omega}_t^i, \mathbf{a}_t^i\}_{i=1}^Q}_{Q(3+9+3+3+4)} \right] \in \mathbb{R}^{6+22Q} \quad (4.7)$$

Here $\mathbf{e}_t^P = (\mathbf{p}_{\text{des},t}^P - \mathbf{p}_t^P) / \max(\|\mathbf{p}_{\text{des},t}^P - \mathbf{p}_t^P\|, 1)$ is the normalized payload tracking error and $\mathbf{v}_t^P$ its velocity. Giving the error as observation instead of an absolute position allows us to give a position setpoint and track trajectories. We normalize the error to have the policy automatically generalize to a target at any distance. For each quadrotor i , $\mathbf{r}_t^i = \mathbf{p}_t^i - \mathbf{p}_t^P$ denotes its position relative to the payload, $\text{vec}(\mathbf{R}_t^i) \in \mathbb{R}^9$ the row-major flattening of its rotation matrix, $\mathbf{v}_t^i$ is the linear velocity in world frame, $\boldsymbol{\omega}_t^i$ its body-frame angular velocities and $\mathbf{a}_{t-1}^i$ the agents previous action. By construction, this global observation vector contains all information required to compute rewards and termination conditions. It can be easily adjusted to include additional information or other mappings.

Local Observations

Since we want to train our policy in a CTDE fashion, we map the global observation vector to each agent's local observation space. During decentralized execution, each agent i receives only the subset of entries corresponding to the payload terms, its own dynamic state block, and the other quadrotors' relative positions. Concretely, agent i observes

$$\mathbf{o}_t^i = \left[\mathbf{e}_t^P, \mathbf{v}_t^P, \mathbf{r}_t^i, \text{vec}(\mathbf{R}_t^i), \mathbf{v}_t^i, \boldsymbol{\omega}_t^i, \mathbf{a}_{t-1}^i, \{\mathbf{r}_t^j\}_{j \in \mathcal{Q} \setminus \{i\}} \right] \in \mathbb{R}^{6+22+Q-1}, \quad (4.8)$$

so that each policy conditions on the payload’s error and velocity, the agent’s full pose and velocity, and its previous action. This restriction enforces the Dec-POMDP structure, by hiding other agents’ internal states, coordination must emerge through the shared reward alone.

The observation structure can also be used for a scenario without a payload, where the team observations containing payload position and velocity are omitted.

Observation Noise

To account for sensor imperfections, we can inject structured Gaussian noise into the global observation vector at each timestep. Concretely, given an observation $\mathbf{o}$, we draw a standard normal vector $\boldsymbol{\eta} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ of the same dimension and compute

$$\mathbf{o}' = \mathbf{o} + \sigma_{\text{obs}} \boldsymbol{\Lambda} \boldsymbol{\eta}, \quad (4.9)$$

where σ_{obs} is a tunable noise amplitude and $\boldsymbol{\Lambda} \in \mathbb{R}^{6+22Q}$ is a diagonal scaling vector to tune the noise level for each observation component. We chose each scale factor by examining the typical range of values for its corresponding observation component and setting the noise magnitude accordingly. This ensures that each component of the observation is corrupted by noise proportional both to its typical magnitude and the global noise level.

4.2.3 Action Space

The policy of each agent i outputs a four-dimensional vector $\mathbf{a}_t^i \in [-1, 1]^4$, representing normalized thrust commands for its rotors. By adopting the SRT model, we retain the full dynamic fidelity of the quadrotor, enabling capable agile maneuvers. These are mapped to physical rotor forces via

$$f_{j,t}^i = \frac{a_{j,t}^i + 1}{2} f_{\max}^i, \quad j = 1, \dots, 4, \quad (4.10)$$

where $f_{\max}^i$ is the agent-specific maximum thrust, randomly perturbed at the start of each episode to improve robustness. Optionally, zero-mean Gaussian noise proportional to $f_{\max}^i$ may be added to each $f_{j,t}^i$ to model actuator uncertainty. We always constrain $\mathbf{a}_t^i \in [-1, 1]^4$ and sample $f_{\max}^i$ within a known bound to produce safe thrust profiles. The resulting thrust vector $\mathbf{f}_t^i$ is then applied in the physics simulator with the proportional torque component.

4.2.4 Randomization

To train a robust policy that can handle a wide range of initial conditions and disturbances, we apply several randomization techniques during training.

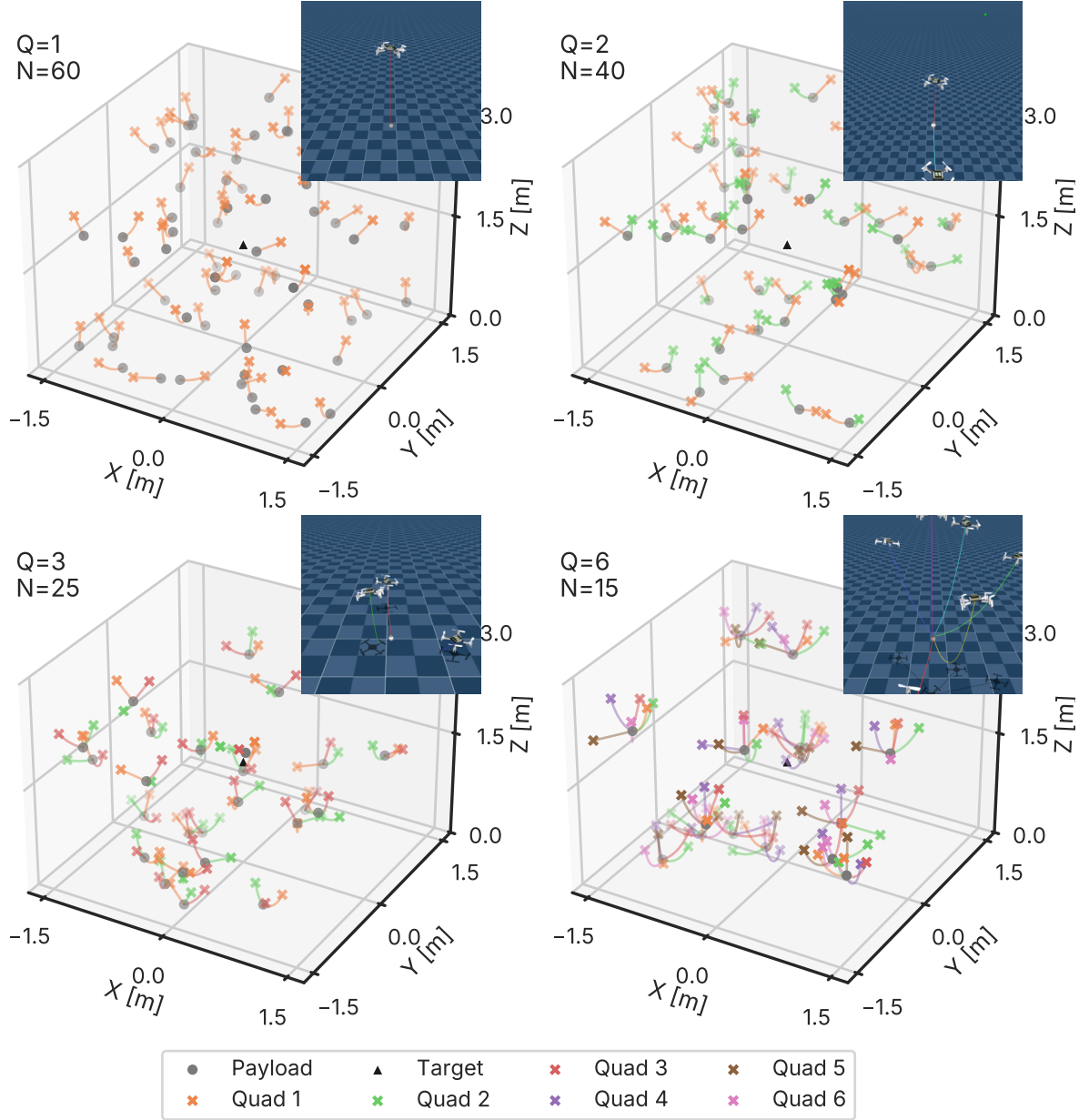


Figure 4.2: Randomly sampled initial configurations for the payload and quadrotors. We show configurations for $Q = 1, 2, 3$, and 6 quadrotors. For visualization purposes, we reduce the number of different runs (N) visualized. Here, the payload position is sampled uniformly without target bias. The payload is positioned at the center of each configuration, while quadrotors are uniformly distributed in a spherical shell around it. A lot of configurations start with slack cables. Some configurations start on the ground.

Initial-State Randomization

A key aspect of our approach is the initial-state randomization $s_0 \sim \rho_0$, which generates diverse initial configurations for the payload and quadrotors. This helps the policy learn to handle a wide range of challenging scenarios, including starting from the ground, hovering, disturbance rejection, modeswitch of the payload cable, and strong payload swing. The initial conditions are sampled at each reset of the environment, ensuring that the policy is exposed to a variety of situations during training. This is crucial for learning robust behaviors. At each reset, the payload center $\mathbf{p}^P$ is sampled uniformly around the target at $[0, 0, 1.5]$ in

$$\mathbf{p}_{xy}^P \sim \mathcal{U}([-1.5, 1.5]), \quad p_z^P \sim \mathcal{U}([-1, 3]), \quad (4.11)$$

where the z -coordinate is clipped to ensure the payload is above the ground, allowing us to adjust the ratio of ground-takeoff starting positions. We additionally add a subset initializations near the payload with $\mathbf{p}^P \sim \mathcal{N}(\mathbf{0}, 0.02\mathbf{I})$. This ratio of target initializations is usually set to 20% in training to help with learning accurate tracking.

We can then use the payload position $\mathbf{p}^P$ as the center point and randomly sample Q quadrotors in a spherical shell around the payload. The radius r_i of each quadrotor is sampled from a normal distribution with mean μ_r and standard deviation σ_r , and then clipped to cable length. The angles θ_i and ϕ_i are sampled from normal distributions with means μ_θ and ϕ_{offset} respectively, and standard deviations σ_θ and σ_ϕ . The angles are then adjusted to ensure they are within the range $[0, 2\pi]$. The equations for sampling the quadrotor positions are as follows:

$$r_i = \text{clip}(\mu_r + \sigma_r \varepsilon_i^{(r)}, r_{\min}, r_{\max}), \quad \theta_i = \mu_\theta + \sigma_\theta \varepsilon_i^{(\theta)}, \quad \phi_i = \frac{2\pi(i-1)}{Q} + \phi_{\text{offset}} + \sigma_\phi \varepsilon_i^{(\phi)}, \quad (4.12)$$

where we found these parameters to work well in practice across different numbers of quadrotors. c is the length of the cable, which is also the maximum radius for the quadrotors.

$$\begin{aligned} \varepsilon_i^{(r)}, \varepsilon_i^{(\theta)}, \varepsilon_i^{(\phi)} &\sim \mathcal{N}(0, 1), \quad \phi_{\text{offset}} \sim \mathcal{U}(-\pi, \pi), \\ \mu_r &= c, \quad \sigma_r = \frac{c}{3}, \quad r_{\min} = 0.05, \quad r_{\max} = c, \\ \mu_\theta &= \frac{\pi}{7}, \quad \sigma_\theta = \frac{\pi}{8}, \quad \sigma_\phi = \frac{\pi}{N+1}. \end{aligned} \quad (4.13)$$

These are converted to Cartesian positions with the z -coordinate clipped.

$$\mathbf{q}_i = \mathbf{p}^P + r_i \begin{bmatrix} \sin \theta_i \cos \phi_i \\ \sin \theta_i \sin \phi_i \\ \cos \theta_i \end{bmatrix}, \quad (4.14)$$

We oversample configurations with this strategy and filter out all configurations where the quadrotors are initially in collision. This results in an even distribution of valid initial configurations with some very harsh initial conditions for any number of quadrotors as shown in Figure 4.2. This also includes starting from the ground.

We also randomize the quadrotor's attitude. Roll and pitch angles are drawn from $\mathcal{N}(0, 10^\circ)$ and clipped to $\pm 90^\circ$, while yaw is sampled uniformly from $[-180^\circ, 180^\circ]$.

Each component of the linear velocity vector $\mathbf{v}$ is sampled from a zero-mean Gaussian with a standard deviation of 0.2 m/s . Each component of the angular velocity (body-rate) vector $\boldsymbol{\omega}$ is sampled similarly with a standard deviation of $20^\circ/\text{s}$.

This creates a wide range of initial conditions, including starting from the ground, hovering, and various disturbances. A lot of them are very challenging to recover from. The policy must learn this recovery behavior. The Gaussian shaping of the randomization parameters is chosen to ensure that there are also sufficient easier initial conditions, such that the policy can learn the basic hover behavior.

Domain Randomization

To further improve robustness, we apply domain randomization techniques during training. For most experiments, we focus on randomizing the quadrotor’s maximum thrust $f_{\max}^i$ within a range of $[0.8, 1.0]$ times the nominal value, with some small Gaussian offset for each motor. This not only ensures that the policy can adapt to variations in the quadrotor’s battery level and motor performance, but also brings a certain level of generalization to payload and quadrotor mass.

We also provide the option to randomize the payload mass, cable length, and other parameters, but these were not used in our experiments, since recent work [12] has shown that zero-shot transfer without domain randomization is possible for the Crazyflie platform. Future work could explore the impact of these additional randomizations on policy robustness, generalization, and sim-to-real transfer.

4.2.5 Reward Design

A key aspect of our approach is a modular reward design for multiple quadrotors with and without cable-suspended payloads. We initially replicated reward structures from prior work for single quadrotors and swarms of quadrotors. The usual reward structure is a tracking reward, penalizing or rewarding the distance to the target, and varying terms to encourage learning of stable flight. The most common ones are an attitude reward component, an angular velocity, and a linear velocity penalty. Depending on the application, this is also extended with a collision penalty and other auxiliary reward terms [11, 12, 31, 32].

Through an iterative process, experimenting with different extensions of these core terms, we found a very effective reward structure. The final reward structure is modular, allowing for easy adjustments and extensions. It consists of several components that encourage the desired behaviors in the quadrotors while maintaining a stable formation and effectively transporting the payload. The goal of our reward is to encourage flight behaviors that recover the cable-suspended payload to its target at a safe speed, while simultaneously minimizing payload swing and vehicle tilt, maintaining a taut suspension cable, enforcing safe spacing between quadrotors, and promoting gentle, low-frequency thrust commands for robust sim-to-real transfer.

Harsh penalties can induce learning-to-terminate behaviors or unstable gradients. In order to prevent this, we shape our unbounded errors inside a smooth, bounded envelope with

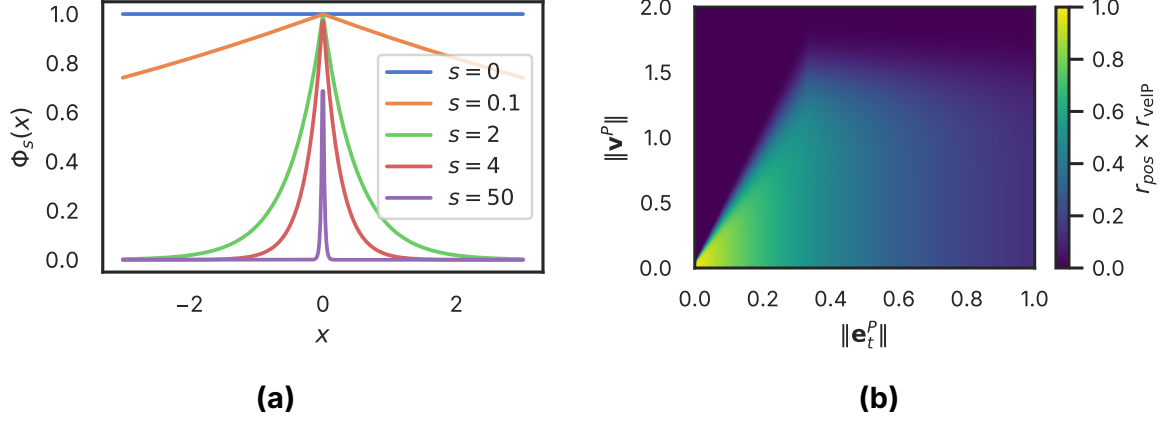


Figure 4.3: (a) The reward shaping function Φ_s used to shape reward components. The steepness factor s determines the decay rate of the function. (b) Plot visualization of the product of payload position and velocity reward. The reward is highest when the payload is close to the target and has very low velocity. A maximum velocity of 1.5m/s is encouraged.

the exponential function instead of penalties. This is a common reward shaping technique in RL, already applied in work on single quadrotors like [9].

$$\Phi_s(x) = \exp(-s|x|), \quad \Phi(x) = \Phi_2(x). \quad (4.15)$$

This keeps the magnitude bounded and makes the derivative taper off rather than spike. Figure 4.3 visualizes this function, showing how it rewards values close to zero. Here s is a scaling factor that determines the steepness of the exponential decay. This function is bounded between 0 and 1, ensuring that the reward remains positive and bounded. This function has a strong peak at 1, decaying rapidly. The exponential decay ensures that the reward approaches zero as the error increases, but never becomes negative. This allows us to use the same reward structure for both positive and negative errors, without introducing instability in the learning process. A major benefit of this design is that it reduces the need for coefficient balancing in the reward function, as is common in previous methods. Our reward consists of the following components:

Payload Position Error

We define the payload position error $\mathbf{e}^P = \mathbf{p}^P - \mathbf{p}_{\text{des},t}^P$. The main objective is to minimize this error, so we define a payload tracking reward

$$r_{\text{pos}} = \Phi(\|\mathbf{e}^P\|) \quad (4.16)$$

We now add multiple auxiliary reward terms to aid learning of hover and stable safe behaviors.

Payload Velocity Alignment

To guide the quadrotor to the target, we add an auxiliary velocity alignment reward.

$$\mathbf{v}_{\text{dir}} = \begin{cases} \frac{\mathbf{v}^P}{\|\mathbf{v}^P\|}, & v > \varepsilon, \\ \mathbf{0}, & \text{else} \end{cases} \quad s_{\text{align}} = \min(40 \|\mathbf{e}^P\|, 2), \quad \mathbf{e}_{\text{dir}} = \frac{\mathbf{e}^P}{\|\mathbf{e}^P\| + \varepsilon}, \quad (4.17)$$

$$r_{\text{dir}} = \Phi_{s_{\text{align}}}(1 - \mathbf{v}_{\text{dir}} \cdot \mathbf{e}_{\text{dir}}). \quad (4.18)$$

This reward encourages the quadrotors to align their velocity with the direction of the payload error, which helps to reduce the tracking error more quickly in a straight line trajectory. We shape Φ with a steepness factor s_{align} that decreases closer to the target. This leads to the direction guidance being more pronounced at larger distances.

We combine this velocity guidance with the payload position error to form a combined tracking reward

$$r_{\text{track}} = (r_{\text{pos}} + r_{\text{dir}})/2 \quad (4.19)$$

We then define multiple stability and safety rewards to ensure the quadrotors maintain a stable flight and avoid collisions with each other.

Velocities

To encourage the quadrotors to maintain a stable velocity, we discourage large velocities of the quadrotors and the payload. We define a maximum desired velocity v_{max} for the payload and quadrotors, allowing us to define a safe velocity range for the payload and quadrotors. We shrink this range as the payload quadrotors approach the target, to ensure they slow down as they get closer. The maximum velocity is set to $v_{\text{max}} = 2 \text{ m/s}$ in our experiments. The velocity reward for the payload is defined as

$$r_{\text{velP}} = \exp\left(-\left(\frac{\|\mathbf{v}^P\|}{(\min(3d, 1) + 0.02) 0.75v_{\text{max}}}\right)^8\right) \quad (4.20)$$

For the quadrotors' velocity, we apply the same shaping and calculate the mean over all quadrotors.

$$r_{\text{velQ}} = \frac{1}{Q} \sum_{i=1}^Q \exp\left(-\left(\frac{\|\mathbf{v}^i\|}{(\min(3d, 1) + 0.02) v_{\text{max}}}\right)^8\right) \quad (4.21)$$

Figure 4.3 shows the interplay of the payload position and velocity reward. The product of these two rewards is essential for good tracking performance. Their combination allows the policy to learn perfect deceleration and stopping at the target, while also keeping velocities in a safe range.

Yaw Rate

We want to strongly discourage large yaw rates, as they can lead to unstable flight and payload swing. We define a yaw rate reward as

$$r_{yaw} = \frac{1}{Q} \sum_{i=1}^Q \phi(\omega_z^i). \quad (4.22)$$

Inter-Agent Safe-Distance

For $Q > 1$ quadrotors, we define a reward that encourages a safe distance between quadrotors. We take the mean of the shaped pairwise distances between all quadrotors, normalized to a maximum of 1. The reward is 0 for our minimum distance of $d_{min} = 0.15m$, which is the minimum safe distance we want to enforce between quadrotors. We then increase the reward to 1 when we reach our safe distance of $d_{safe} = 0.18m$. The reward is defined as

$$r_{dist} = \begin{cases} \frac{1}{Q(Q-1)} \sum_{i \neq j} \text{clip}\left(\frac{1}{d_{safe} - d_{min}} \|\mathbf{p}^i - \mathbf{p}^j\| - d_{min}, 0, 1\right), & Q > 1, \\ 1, & Q = 1. \end{cases} \quad (4.23)$$

Orientation

We calculate the tilt angle θ^i between each quadrotor's body-frame z-axis and the world z-axis. We take the mean over all quadrotors and apply a shaping function Φ to discourage large roll or pitch angles that can lead to instability and payload swing. The reward is defined as

$$r_{up} = \frac{1}{Q} \sum_{i=1}^Q \Phi(\theta^i). \quad (4.24)$$

Taut-String Maintenance

We want to encourage the quadrotors to keep the cables taut, to remain in control of the payload. We therefore encourage an increase in the distance of the quadrotors to the payload. Let $d^{P,i} = \|\mathbf{p}^i - \mathbf{p}^P\|$ and $h^i = (\mathbf{p}^i)_z - (\mathbf{p}^P)_z$. A taut cable of length L is encouraged by maximizing both terms via

$$r_{taut} = \frac{1}{L} \left(\frac{1}{Q} \sum_{i=1}^Q d^{P,i} + \frac{1}{Q} \sum_{i=1}^Q h^i \right). \quad (4.25)$$

Collision and Boundary Penalties

We penalize collisions between quadrotors and the ground. Additionally, we define some out-of-bound conditions for large payload error and very large quadrotor angular velocities. With the indicators $\mathbb{I}_{coll}$, $\mathbb{I}_{oob}$ we simply calculate a penalty for collisions and out-of-bounds conditions as

$$r_{coll} = -10, \mathbb{I}_{coll}, \quad r_{oob} = -10 \mathbb{I}_{oob}. \quad (4.26)$$

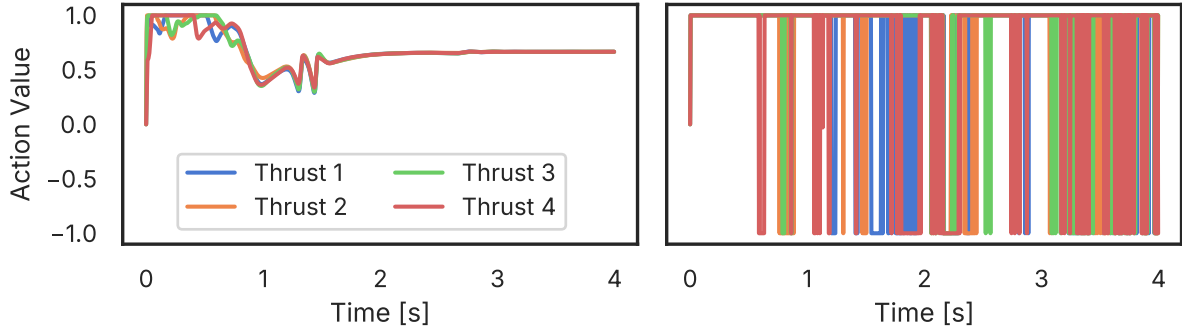


Figure 4.4: Comparison of policy trained with and without the action regularization reward. The left plot shows the actions over time of the policy with regularization, while the right plot shows the actions of the policy without regularization. The regularized policy produces smoother and more stable actions, while the unregularized policy exhibits large fluctuations in thrust commands jumping between the action limits.

Smoothness and Energy Cost

With high-frequency continuous control tasks, we often observe that the policy learns to produce very aggressive actions, saturating the action range. To encourage smoother actions and reduce energy consumption, we introduce two regularization terms. One discourages actions at the limits of the action range, while the other penalizes large changes in rotor thrust and variations between rotors. Let $\mathbf{a}_t^i, \mathbf{a}_{t-1}^i$ be consecutive normalized thrust vectors. The action smoothness reward is calculated as the difference between consecutive actions and the mean thrust of the four rotors

$$r_{\text{smooth}} = 20 \left(\frac{1}{Q} \sum_{i=1}^Q \|\mathbf{a}_t^i - \mathbf{a}_{t-1}^i\|_1 + \frac{1}{Q} \sum_{i=1}^Q \|\mathbf{a}_t^i - \bar{\mathbf{a}}_t^i \mathbf{1}\|_1 \right), \quad \bar{\mathbf{a}}_t^i = \frac{1}{4} \sum_{j=1}^4 a_{t,j}^i \quad (4.27)$$

This term encourages smooth, low-frequency thrust commands and penalizes large changes in rotor thrust. Additionally, we penalize actions that are close to the action limits, which can lead to aggressive behavior and instability. We define the energy cost as

$$r_{\text{energy}} = \frac{1}{4Q} \sum_{i=1}^Q \sum_{j=1}^4 \exp(-50 |a_{t,j}^i|) + \exp(50 (a_{t,j}^i - 1)) \quad (4.28)$$

The first term discourages abrupt changes in thrust, while the second term penalizes high rotor speeds, which can lead to excessive energy consumption and instability. The exponential shaping ensures that the penalty is more pronounced for larger deviations, promoting smoother actions.

Figure 4.4 shows the difference in actions between a policy trained with and without these two action regularization terms. The policy with regularization produces smoother and more stable actions, while the unregularized policy exhibits large fluctuations in thrust commands, jumping between the action limits. This regularization is crucial for stable training and sim-to-real transfer.

Aggregate Reward

We combine the individual reward components into a single reward signal for each agent. The reward components for stability are grouped as

$$r_{\text{stab}} = \frac{1}{5}(r_{\text{velP}} + r_{\text{velQ}} + r_{\text{yaw}} + r_{\text{up}} + r_{\text{taut}}). \quad (4.29)$$

The safety components that should always be enforced are grouped as

$$r_{\text{safety}} = \frac{1}{5}(r_{\text{coll}} + r_{\text{oob}} + r_{\text{smooth}} + r_{\text{energy}} + r_{\text{dist}}). \quad (4.30)$$

Finally, we combine the tracking reward with the stability and safety components to form the overall reward

$$r = r_{\text{track}} r_{\text{stab}} + r_{\text{safety}}. \quad (4.31)$$

This product structure ensures the policy learns to track the payload while constantly maintaining stable formations. This is important because the simultaneous optimization of position error and velocities can lead to policies getting stuck in local minima, with a static payload position error remaining. The additive safety penalties strictly discourage unsafe or aggressive behaviors.

This reward formulation naturally scales with the number of quadrotors Q , retaining its structure and incentives as Q varies. It even works in the simplified case of a single quadrotor without a payload. We simply set $r_{\text{taut}} = 1$ and use the quadrotor position for tracking error calculation. We still obtain a coherent reward that still encourages stable, smooth flight and graceful control even in the absence of cable-tautness considerations.

The reward structure can also be adapted to other tasks by replacing the tracking reward component with other task-specific rewards. This makes the reward a good basis for a wide range of scenarios with multiple quadrotors. We can now apply our reward function to train policies for different numbers of quadrotors with and without payload.

4.3 Training

Our training pipeline follows the approach of [60], leveraging the JaxMARL framework as a basis for our training method. We use the IPPO algorithm, which extends PPO to multi-agent settings by optimizing a joint policy across all agents while maintaining decentralized execution. We also investigated the use of MAPPO with a centralized critic, but found that IPPO performs sufficiently for few quadrotor scenarios.

The decentralized policies are learned end-to-end via IPPO within our highly parallelized JAX/MJX simulation environment. At each iteration, a fixed number of synchronous actors collect trajectories in N parallel environments over T time steps, yielding NT experiences per update. Collected rewards are bootstrapped with the learned critic to compute targets, and advantages are estimated using GAE with discount factor γ and smoothing parameter λ . The overall loss combines the clipped policy objective

$$L_{\text{PPO}}(\theta) = \mathbb{E}[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t)], \quad (4.32)$$

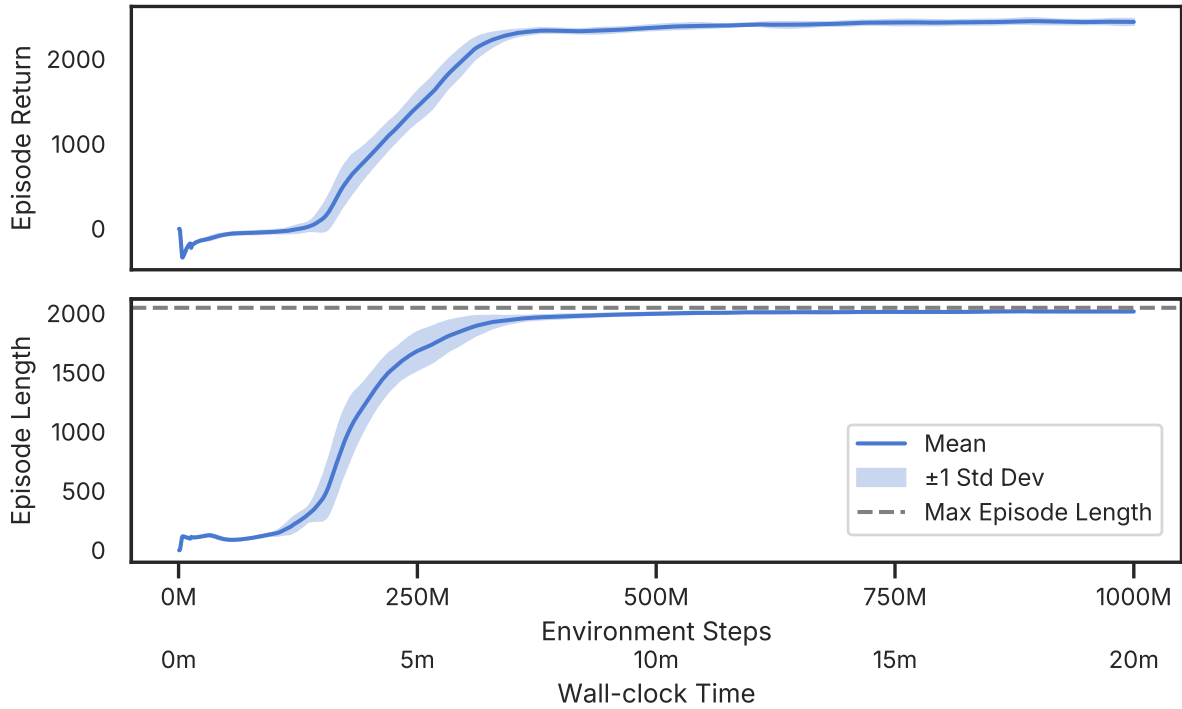


Figure 4.5: Average training metrics for one quadrotor with payload scenario. The top plot shows the average episode returns, and the bottom plot shows the average episode lengths over environment steps and wall-clock time. The shaded area represents the standard deviation across 10 seeds. The maximum possible episode length here is 2048 environment steps, which corresponds to 8.2 s of flight time. In the end, nearly all episodes are completed, indicating stable flight.

a mean-squared value-function error, and an entropy regularizer to encourage sufficient exploration. Gradient norms are clipped to enhance numerical stability.

Since our environment has a certain level of complexity, we always train on a high number of parallel environments (16384). This might not be very sampling efficient, but it allows us to collect a large number of experiences with sufficient variance of configurations in a short time, which is crucial for stable training of policies. A lower number of environments also usually trains a working policy, but the training progress has higher variance, and it is more likely to get stuck in local minima for position tracking. In each environment, we only take 32 steps per episode. Giving us 128 minibatches of size 4096 for each episode. The training is performed on a single GPU, which allows us to leverage the high-performance capabilities of JAX and Brax for efficient training. Figure 4.5 shows the training progress of the two quadrotor policies.

The training process and hyperparameters are virtually the same for all numbers of quadrotors, only the policy size is slightly adjusted for more quadrotors. We train the policy on a single RTX Ada 4000 GPU with 20GB of RAM. This gives us training speeds of up to 1 million environment steps per second, depending on the number of parallel environments and Quadrotors. We trained the policies in our experiments on one billion environment steps, which corresponds to roughly 10 minutes of training time for the single quadrotor task, 20 minutes for the dual quadrotor task, and 60 minutes for the three quadrotor tasks. We are also able to train a single quadrotor policy without a payload in less than 3 minutes.

4.3.1 Policy Architecture

Both actor and critic networks are instantiated as fully connected, feed-forward multilayer perceptrons. The actor network comprises an input layer matching the local observation dimension, three hidden layers of 64 units with tanh activations, and a linear output layer producing the action-mean vector. A learned log-standard deviation vector of the same dimensionality parameterizes a diagonal Gaussian policy. The critic network features three hidden layers of 128 tanh units each, terminating in a scalar output. All dense layers employ orthogonal weight initialization and zero biases. During training and rollouts, actions are sampled stochastically from the policy distribution. At evaluation time, the deterministic mean action is used.

4.3.2 Hyperparameters

We run a Bayesian optimization sweep, with early termination of runs, over the hyperparameters to find the best configuration for our task. We first conduct a coarse search over a wide range of values, then refine the search around the most promising regions. The final hyperparameters are selected based on the best-performing policy with the highest reward. Table 4.1 summarizes the key hyperparameters used in our experiments.

Table 4.1: Key hyperparameters for training.

Hyperparameter	Value
Total environment timesteps	1×10^9
Number of environments	16,384
Number of steps per update	32
Number of minibatches	128
Update epochs	8
Anneal learning rate	false
Learning rate	4×10^{-4}
Activation function	tanh
Entropy coefficient	0.01
Discount factor (γ)	0.995
GAE lambda (λ)	0.95
Actor network architecture	[64, 64, 64]
Critic network architecture	[128, 128, 128]

5. Evaluation and Experiments

This chapter describes the evaluation methodology and outlines the metrics and figures that we used to quantify the performance of our learned policies on both single-quadrotor and multi-quadrotor cable-suspended payload transport tasks.

5.1 Experimental Setup

We first evaluate the performance of our learned policies in our simulation environment. We use only the actor network for evaluation, as the critic is not needed. The policies are trained using the IPPO algorithm. All experiments were conducted in simulation using a policy frequency of 250 Hz with one physics step per control action. We set both observation and actuator noise to zero, randomized thrust within 80-100% thrust range, used a 0.3 m cable length, and fixed the payload mass at 0.01 kg.

Two primary tasks were evaluated: the recovery from harsh conditions task and the figure-eight trajectory tracking task. The recovery task involved stabilizing the payload at a target position after being disturbed, while the trajectory tracking task required the quadrotor to follow a precomputed figure-eight path while keeping the payload stable.

In the recovery task, episodes lasted 2,500 steps, the payload target remained fixed at (0, 0, 1.5) m, and the agent began from random, harsh initial states. The payload and quadrotor positions, as well as the quadrotors' orientations and velocities, are randomized. Metrics were gathered over 1,000 rollouts. We evaluate the specific policies for a single quadrotor with payload, teams of two, three, and six quadrotors were evaluated, each initialized as described in section 4.2.4. The payload was always initialized at (0, 0, 1.5) m, and the quadrotors were positioned randomly in a half-sphere around it. It's important to note that this includes different cable modes, with sometimes the cable being slack and sometimes taut, forcing the quadrotor to stabilize the payload in all modes. These states emulate the state after a strong disturbance, like a collision with an obstacle or the quadrotors being thrown into the air.

In the figure-eight tracking task, each episode ran for 5,000 steps, simulating 20 s, the payload has to follow a precomputed figure-eight path, and the quadrotor is always initialized at the center with quads in random positions, with performance aggregated over 100 rollouts.

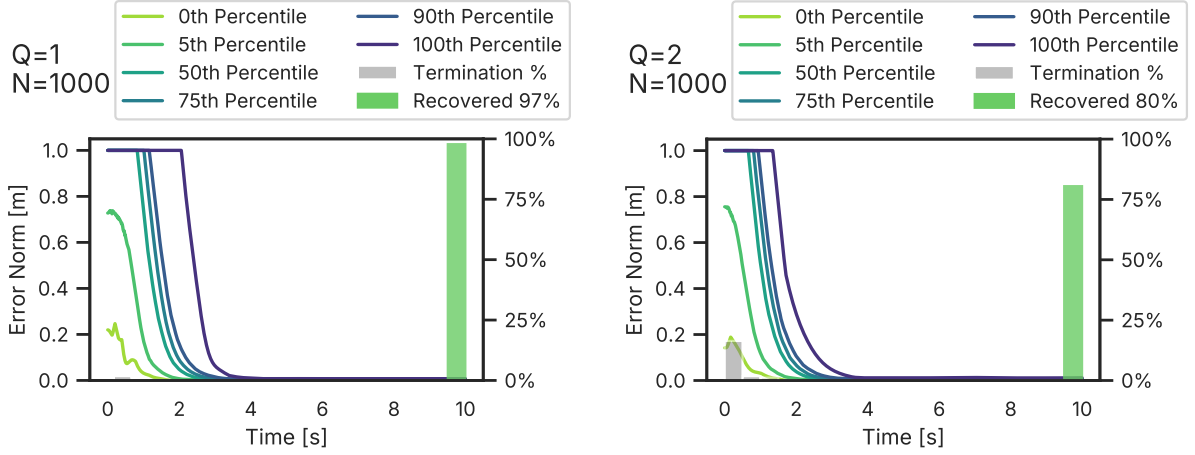
5.2 Single-Quadrotor Payload Control

We first evaluate the learned policy on the single-vehicle case, testing both harsh condition recovery and trajectory tracking on a figure-eight path. We run each with a policy trained specifically for the single quadrotor with payload scenario. The results are shown in Figure 5.1a and Figure 5.2. Figure 5.2 shows some example runs of the recovery task, where the quadrotor is initialized in random states and has to stabilize the payload at the target position $(0, 0, 1.5)$ m. The quadrotor successfully stabilizes the payload within 2 s for most initial conditions. In the figure, the bottom two runs labeled 6 and 7 show takeoff from the ground, which is hard due to a very sudden mode switch from slack to taut cable close to the ground. Runs 1 to 5 show the active swing counteraction of the quadrotor, where it actively stabilizes the payload on the path to the target. Runs 3 and 4 specifically show a very strong initial payload swing. All runs stabilize the payload swing before reaching the target position without any oscillations or overshoot. They then stabilize there without much movement for the rest of the run. We can see that the policy generalizes to all relative positions of the payload to the target and the quadrotor to the payload. During recovery, the payload reaches speeds of up to 1.7 m/s, highlighting the learned agility of the policy. If this level of agility is not desired, the speed can be reduced by adjusting the linear velocity components of the reward function.

We only show recovery from up to 1.5 m. This, though, generalizes to any distance by clipping the length of the position error vector to 1 m. In addition to the qualitative analysis, we also show a more thorough statistical analysis in Figure 5.1a. Here we statistically evaluate 1,000 recovery runs from harsh conditions. The termination histogram shows that 97% of harsh initial conditions are recovered successfully, while the remaining terminate right in the beginning due to very harsh initialization. The position error over time also shows that we recover from distances of up to 1 m in roughly 2 s. It also shows that all runs stably hover at the target for over 5 s after reaching it.

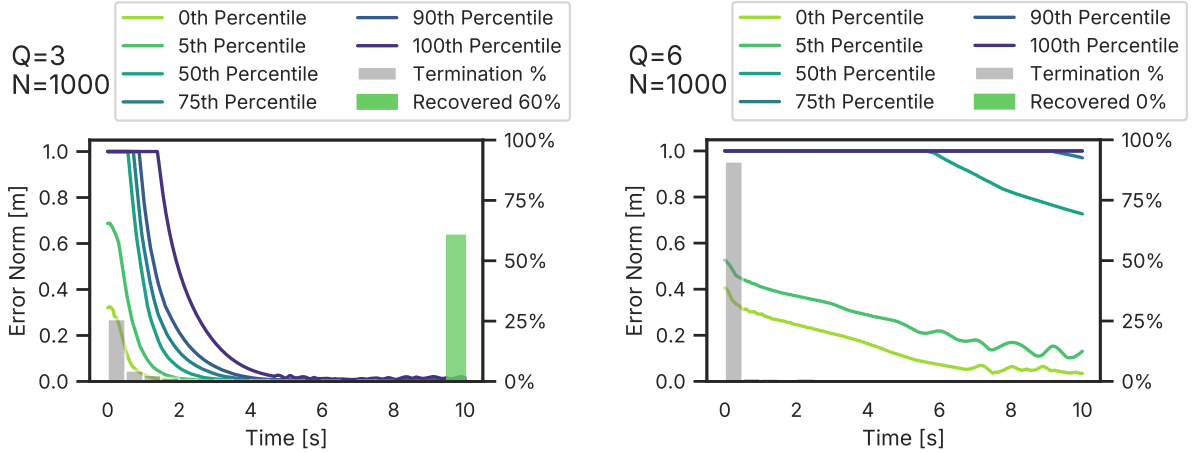
To show that the position tracking capabilities generalize to a moving setpoint, we evaluate on a precomputed figure-eight trajectory. For each timestep, we sample the policy position setpoint on the figure-eight trajectory, leading to the tracking behavior shown in Figure 5.3. The figure shows the path of five runs randomly initialized around the start point of the trajectory, with the quads in suboptimal states. The figure shows how the quads very quickly manage to stabilize themselves and then continue after around 3 s to accurately follow the trajectory. Even though all quads are initialized with slightly different thrust ranges, they all manage to accurately track the correct height of the trajectory. Due to the policy currently just being able to do position tracking, it always slightly lags behind the trajectory. This can be seen at the end of the trajectory, where it does not fully reach the center. This could be reduced by explicitly training on trajectory following and adding multiple trajectory points or velocity states, as shown in previous work.

These experiments show that the trained policy for the single quadrotor with payload case exhibits great payload swing stabilization ability, while also being agile and very accurate in tracking.



(a) Recovery in the single quadrotor with payload scenario from 1000 different initial harsh states. The Percentiles show a smooth reduction of the initial high tracking error, converging to 0. The termination percentage shows that most runs are successful and not terminated.

(b) Recovery in the two quadrotor with payload scenario from 1000 different initial harsh states. The Percentiles show a smooth reduction of the initial high tracking error for most initial harsh states, eventually converging to 0. The termination percentage shows that a high number of runs are successful and not terminated, while most terminations occur right at initialization, because of very harsh initialization.



(c) Recovery in the three quadrotor with payload scenario from 1000 different initial harsh states. The Percentiles show a smooth reduction of the initial high tracking error, converging to 0. The termination percentage shows that nearly all runs are successful and not terminated, while most terminations occur right at initialization, because of very harsh initialization.

(d) Recovery in the six quadrotor with payload scenario from 1000 different initial harsh states. Most runs are terminated early, indicating the limits of the approach.

Figure 5.1: Recovery error over time for different quadrotor configurations with payload. The lines show the distance percentiles of the unterminated runs at each timestep. The green bar indicates the percentage of runs that were successfully recovered and stabilized at the target. The gray bars show the frequency of terminations over time.

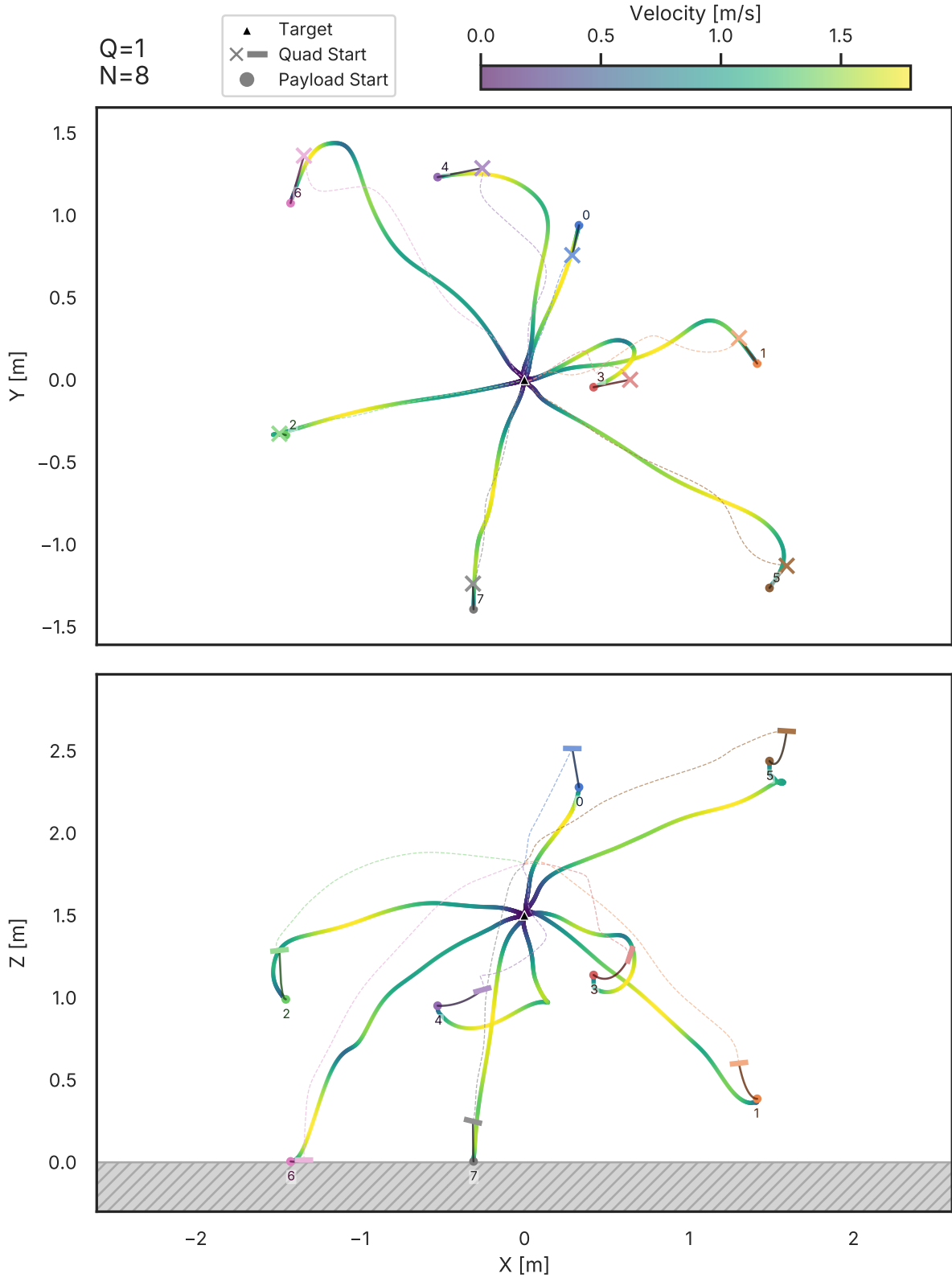


Figure 5.2: Recovery of one quadrotor with payload from harsh conditions. The top plot is a top-down (xy) view, while the bottom one is a side view (xz). The figure shows $N=8$ runs. The position is initialized randomly, and the goal is to recover to the target and stabilize. The quadrotor successfully stabilizes the payload within 2 s. The initial states of the quadrotor and payload are marked and labeled with their run id. The cables are drawn to represent their mode and cable length, slackness in the cable being shown as a curve.

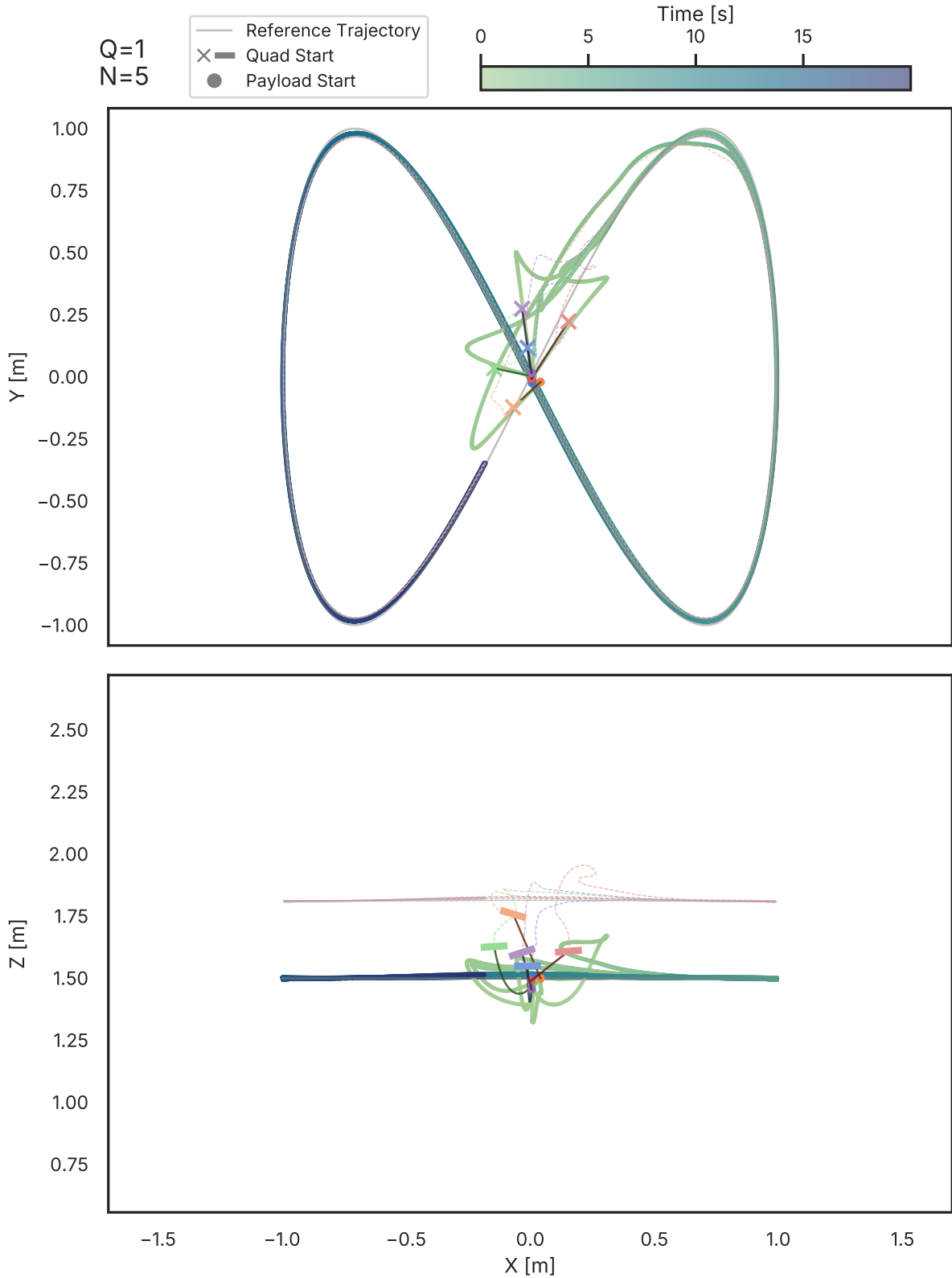


Figure 5.3: Figure-eight trajectory tracking for one quadrotor with payload. The top plot is a top-down (xy) view, while the bottom one is a side view (xz). The figure shows $N=5$ runs. The position is initialized near the target with random quadrotor states. The quadrotor successfully follows the figure-eight path while stabilizing the payload.

5.3 Multi-Quadrotor Payload Transport

We extend the evaluation to cooperative transport tasks involving teams of two and three quadrotors to show learned coordinated behavior. The simulation configuration and scenarios are set up similarly to the single-quadrotor case, with the payload always initialized at $(0, 0, 1.5)$ m and the quadrotors positioned randomly in a half-sphere around it as described in section 4.2.4. The policies are trained specifically for the number of quadrotors in the team. For three quadrotors, we increase the payload mass to 20 g to ensure that the payload is heavy enough to require cooperation. The policies are trained using the IPPO algorithm with the same hyperparameters as in the single-quadrotor case.

5.3.1 Two Quadrotors

We first evaluate the learned policy on the two-quadrotor case, testing both harsh condition recovery and trajectory tracking on a figure-eight path. We run each with the policy trained specifically for the two quadrotors with payload scenario. The results are shown in Figure 5.1b, Figure 5.4 and Figure 5.5.

Figure 5.4 shows some example runs of the recovery task, where the quadrotors and payload are initialized in random states and have to stabilize the payload at the target position. The runs labeled 0 and 1 show successful takeoff. 4 and 3 show stabilization of swinging payload. 4 in specific shows a very harsh tilt orientation of the quadrotor that is recovered from. 3 to 7 are all initialized with slack cables. The payload is stabilized at the target within 2 s, moving at speeds of up to 1.5 m/s. The quadrotors then stabilize the payload at the target position without oscillations or overshoot. We also evaluate this in Figure 5.1b, where we statistically evaluate 1,000 recovery runs from harsh conditions. The termination histogram shows that 81% of harsh initial conditions are recovered successfully, while the remaining terminate mostly in the beginning due to very harsh initialization. The position error over time also shows that we recover from distances of up to 1 m in roughly 2 s. It also shows that all runs stably hover at the target for over 5 s after reaching it. Some of the failed runs struggle to recover and get stuck in an oscillation, leading to termination. This is due to very harsh initialization, leading to the quadrotors running into each other right after initialization. This can be improved by better initialization of the quadrotors and payload, which is discussed in section 4.2.4.

To show that the position tracking capabilities generalize to a moving setpoint, we evaluate on a precomputed figure-eight trajectory. We use the same setup as with the single-quadrotor case. We show the results in Figure 5.5. The figure shows the path of 5 runs randomly initialized around the start point of the trajectory, with the quads in suboptimal states. The figure shows how the quads very quickly manage to stabilize themselves and then continue after around 3 s to follow the trajectory. The tracking is less accurate than the single-quadrotor case, but still shows good performance. Even though all quads are initialized with slightly different thrust ranges, they all manage to accurately track the correct height of the trajectory. We might further improve the tracking performance by explicitly training on trajectory following and adding multiple trajectory points or velocity states, as shown in previous work, and also adding the velocity state of the other quadrotors to the observation space.

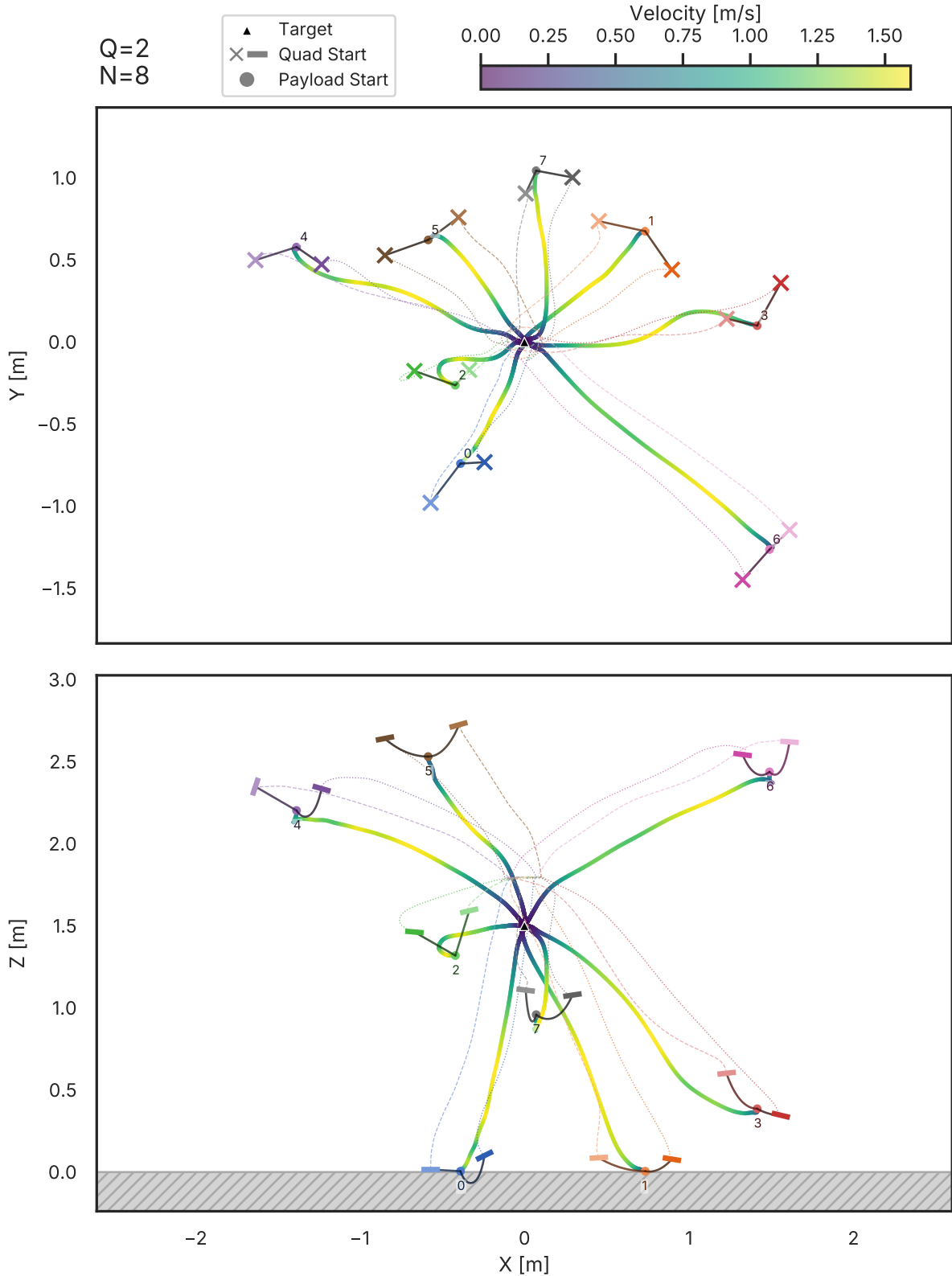


Figure 5.4: Recovery of two quadrotors with payload from harsh conditions. The top plot is a top-down (xy) view, while the bottom one is a side view (xz). The figure shows $N=8$ runs. The position of the payload and the quadrotors is initialized randomly, and the goal is to recover to the target and stabilize. The quadrotor successfully stabilizes the payload within 2 s. The initial states of the quadrotor and payload are marked and labeled with their run id. The cables are drawn to represent their mode and cable length, slackness in the cable being shown as a curve.

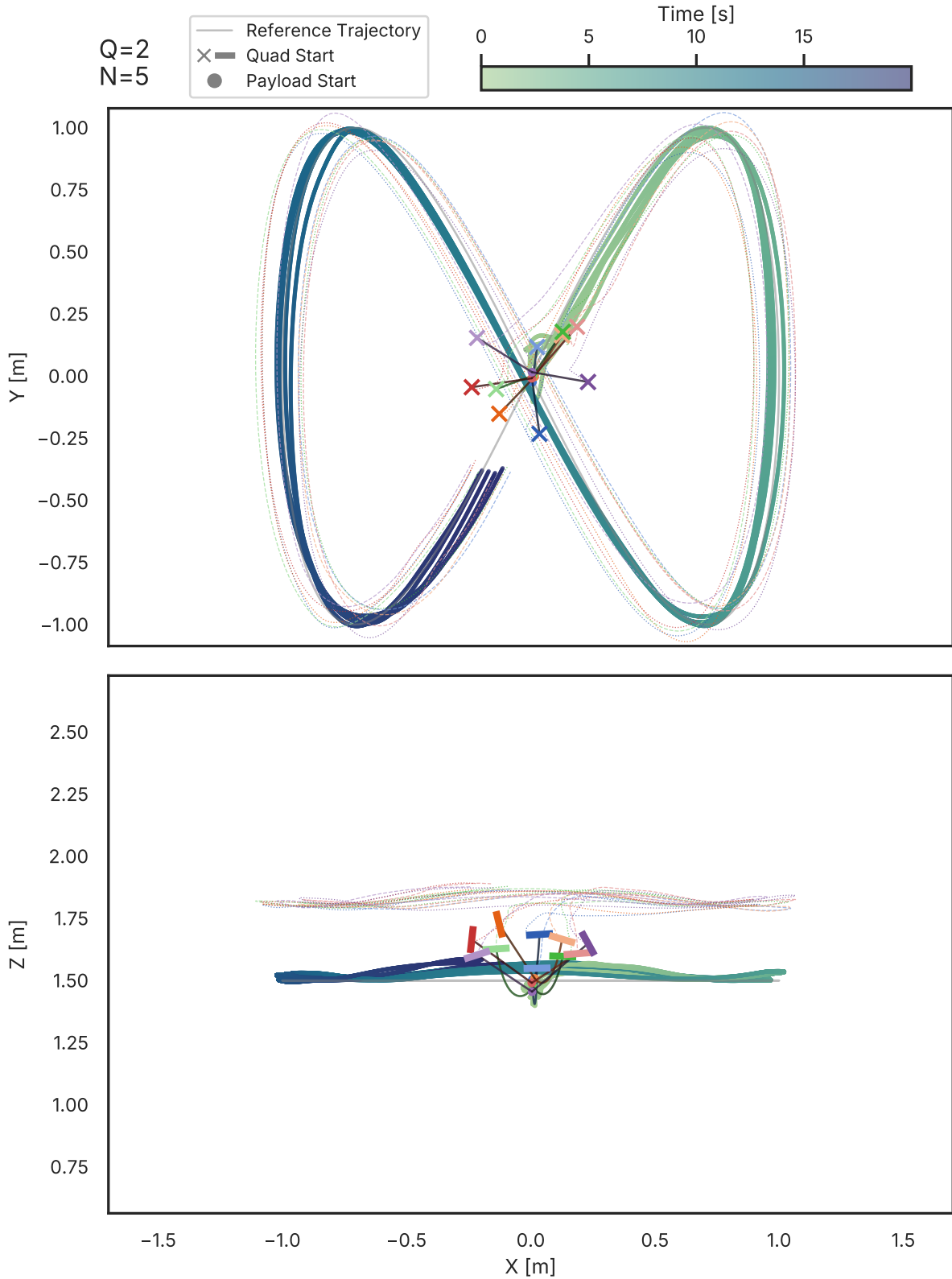


Figure 5.5: Figure-eight trajectory tracking for two quadrotors with payload. The top plot is a top-down (xy) view, while the bottom one is a side view (xz). The figure shows $N=5$ runs. The position is initialized near the target with random quadrotor states. The quadrotor successfully follows the figure-eight path while stabilizing the payload.

5.3.2 Three Quadrotors

We extend the evaluation to three quadrotors, testing both harsh condition recovery and trajectory tracking on a figure-eight path. We run each with the policy trained specifically for the three quadrotor with payload scenario. The results are shown in Figure 5.6 and Figure 5.7. The policies are trained using the IPPO algorithm with the same hyperparameters as in the single-quadrotor case, but with a payload mass of 20 g to ensure that the payload is heavy enough to require cooperation.

Figure 5.6 shows some example runs of the recovery task, where the quadrotors and payload are initialized in random states and have to stabilize the payload at the target position. The payload is stabilized at the target within 2 s, moving at speeds of up to 1.5 m/s. The quadrotors then stabilize the payload at the target position without oscillations or overshoot.

We also evaluate this in Figure 5.1c, where we statistically evaluate 1,000 recovery runs from harsh conditions. The termination histogram shows that 60% of harsh initial conditions are recovered successfully, while the remaining terminate mostly in the beginning due to very harsh initialization. The position error over time also shows that we recover from distances of up to 1 m in roughly 3 s. It also shows that all runs stably hover at the target for over 5 s after reaching it. Some of the failed runs struggle to recover and get stuck in an oscillation, leading to termination. This is due to very harsh initialization, leading to the quadrotors running into each other right after initialization. This can be improved by better initialization of the quadrotors and payload, which is discussed in section 4.2.4.

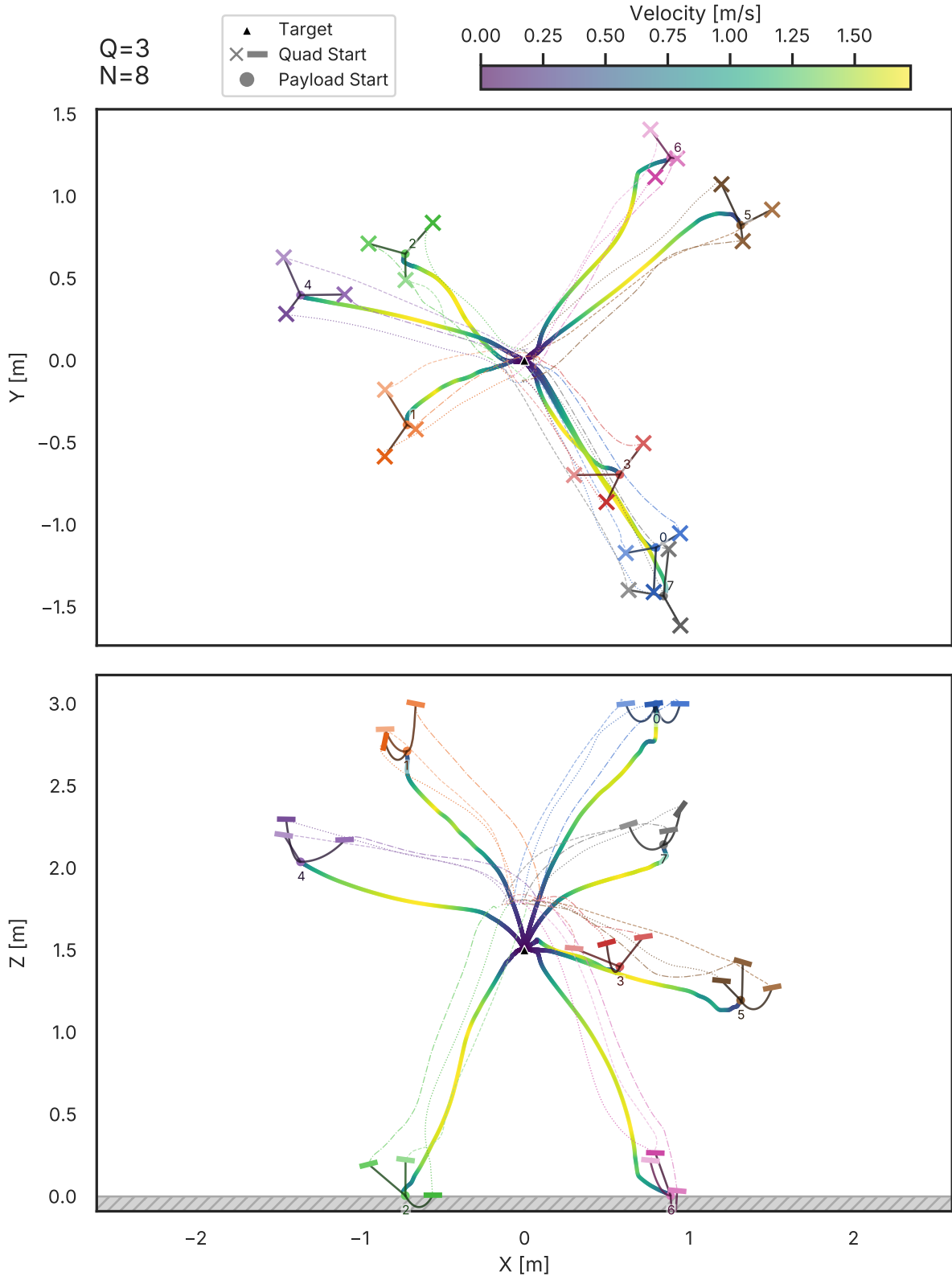


Figure 5.6: Recovery of three quadrotors with payload from harsh conditions. The top plot is a top-down (xy) view, while the bottom one is a side view (xz). The figure shows $N=8$ runs. The position of the payload and the quadrotors is initialized randomly, and the goal is to recover to the target and stabilize. The quadrotor successfully stabilizes the payload within 2 s. The initial states of the quadrotor and payload are marked and labeled with their run id. The cables are drawn to represent their mode and cable length, slackness in the cable being shown as a curve.

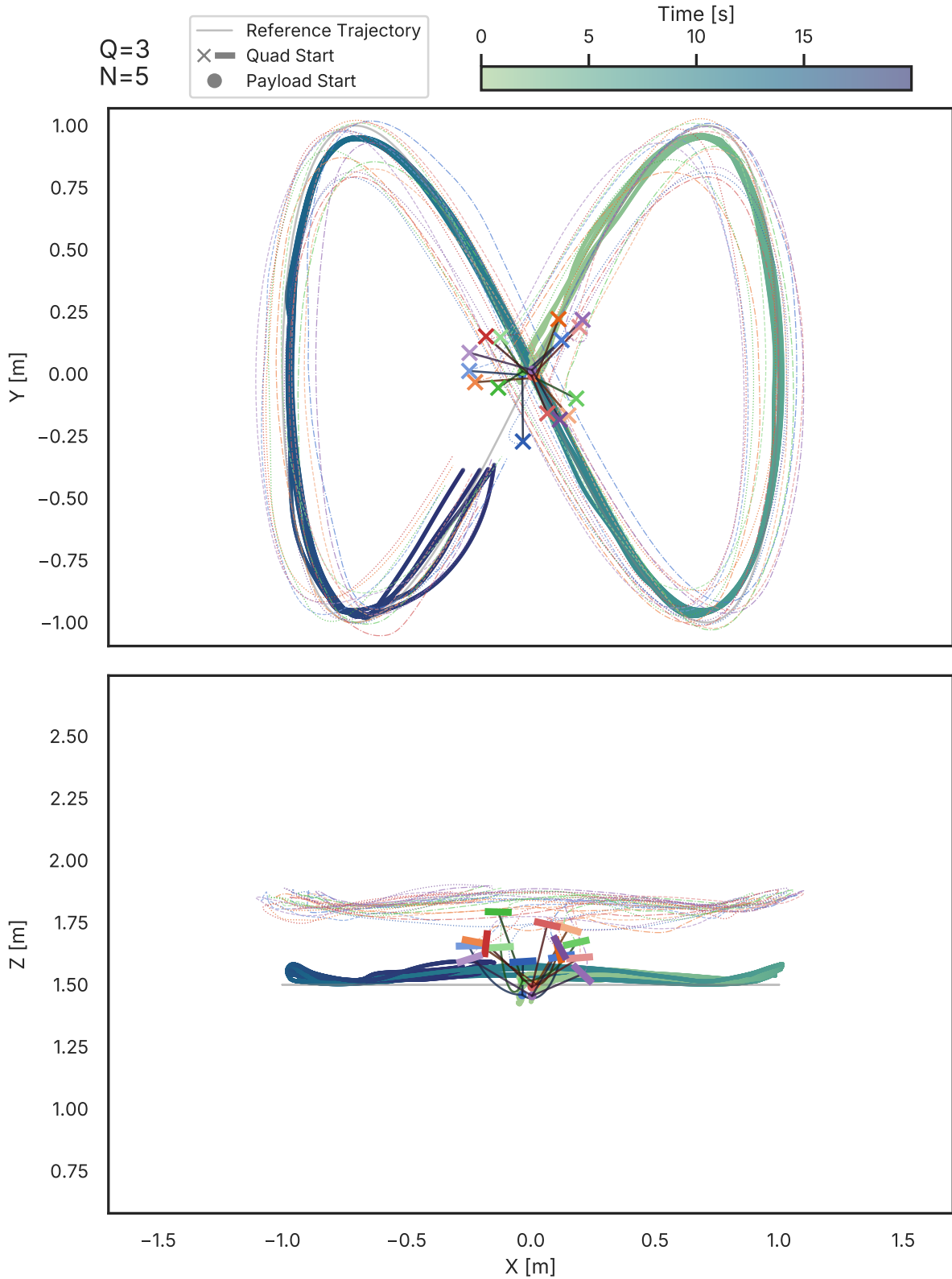


Figure 5.7: Figure-eight trajectory tracking for three quadrotors with payload. The top plot is a top-down (xy) view, while the bottom one is a side view (xz). The figure shows $N=5$ runs. The position is initialized near the target with random quadrotor states. The quadrotor successfully follows the figure-eight path while stabilizing the payload.

5.3.3 Six Quadrotors

The last configuration we evaluate is a team of six quadrotors, testing recovery from harsh conditions to evaluate the scalability of our approach. We run them with the policy trained specifically for the six quadrotor with payload scenario. The results are shown in Figure 5.1d, evaluating 1,000 runs on the recovery scenario.

The policies are trained using the IPPO algorithm with the same hyperparameters as in the single-quadrotor case, but with a payload mass of 40 g to ensure that the payload is heavy enough to require cooperation. The termination histogram in Figure 5.1d shows that most runs are terminated early, indicating the limits of our approach. These terminations mostly happen because the quadrotors struggle to coordinate their movements effectively under the increased complexity of the scenario, leading to collisions. A major limitation of our approach is that we currently do not preprocess or sort the other quadrotors' positions in the observation space, which leads to the policy not being able to effectively coordinate the movements of the quadrotors, because it needs to cover all possible combinations of relative positions.

One possible solution to this could be to sort the other quadrotors' positions by their distance to the agent in the observations, to reduce the complexity. Other approaches that future work could investigate to improve on these limitations include privileged critic approaches, better representation of the payload and quadrotors in the observation space, and more complex network architectures, for example utilizing recurrent networks or attention architectures as in [11].

5.4 Comparison with Baseline

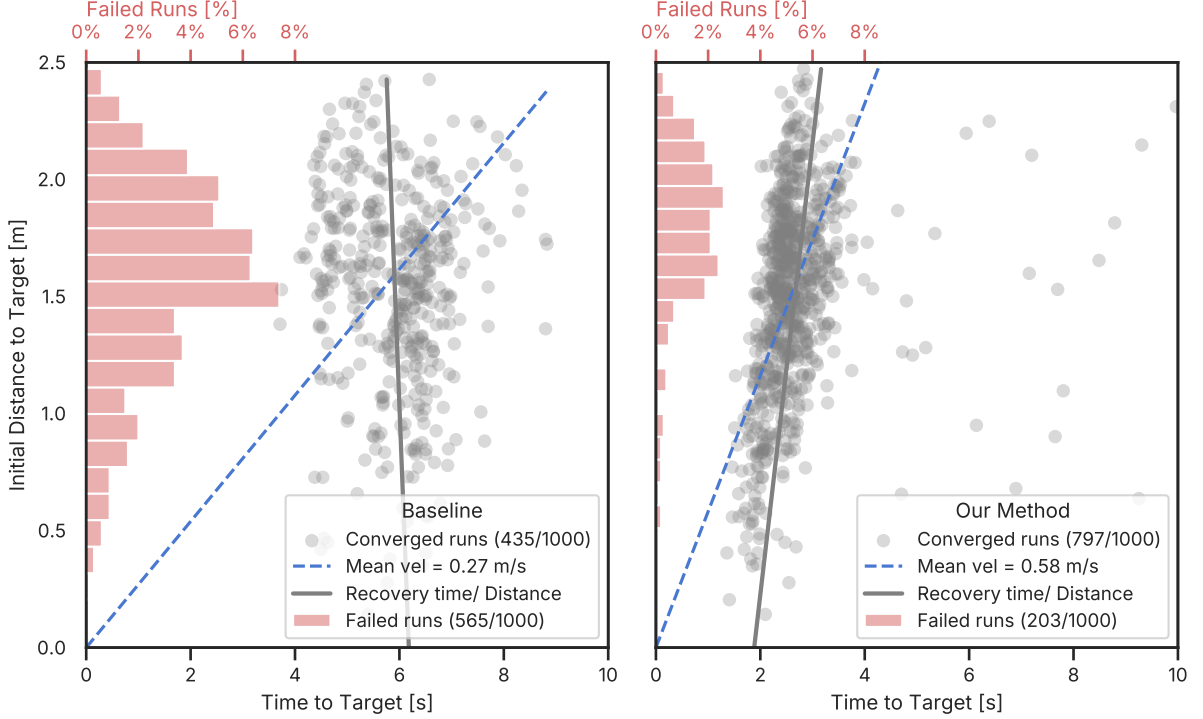


Figure 5.8: Comparison of the learned policy with the baseline method from [13]. We compared the methods on $N=1000$ runs initialized in harsh conditions. The left plot shows the baseline method. The right plot shows the learned policy. Our method achieves a significantly lower recovery time and failure rate.

We compare our learned policy with the baseline method from [13] in terms of recovery from harsh conditions performance. The baseline planner models each cable as a rigid rod and relies on centralized offline trajectory optimization with an online controller that tracks these precomputed paths, which limits its ability to capture cable swing and mode switching, adapt to unexpected disturbances or payload changes or collisions, and meet real-time computational demands. Our method addresses these limitations by leveraging RL for a more robust decentralized solution.

We compare our method with the baseline on the two quadrotor with payload scenario, where we evaluate the recovery from harsh conditions. We initialize the payload at $(0, 0, 1.5)$ m and the quadrotors in random positions around it. The baseline method is run with the same initialization and the same target position. We evaluate both methods on 1000 runs each, with the payload initialized in random states and the quadrotors in random positions around it. To improve the baseline performance, we precompute a straight line polynomial high-order trajectory from the initial position to the target position, which is then tracked by the baseline method.

The results are shown in Figure 5.9 and Figure 5.8. Figure 5.8 shows significantly higher recovery rates of 80% with our method compared to 44% with the baseline method. Our method recovers faster and with less variance in recovery speed. This is largely due to the

policy being able to swiftly reduce payload swing. The baseline method fails to recover from harsh conditions in most cases, while our method is able to recover from most initial conditions and stabilize the payload at the target position.

Figure 5.9 also shows the qualitative difference of the trajectories in the recovery task, where the quadrotors and payload are initialized in random states and have to stabilize the payload at the target position. Run 1 clearly shows how the baseline first drops down before recovering to the target. This is due to the baseline’s reliance on a rigid rod model of the cable. Leading to correction behavior where the controller tries to push the payload in the correct direction. Here, it is also shown that our method achieves a more efficient straight line trajectory to the target position, while the baseline method follows a more complex trajectory spiraling to the target.

It is important to note that the baseline method might perform better in some specific scenarios, especially when the payload is in a very stable state and the quadrotors are in optimal positions. However, our method shows a more robust performance across a wider range of initial conditions and disturbances, making it more suitable for real-world applications where such conditions are common. Also, our method currently does not take obstacles into account, which is a limitation of the current approach. Future work could extend our method or combine it with the motion planning of the baseline method to achieve obstacle avoidance capabilities. We have not yet achieved sim-to-real transfer with our method, which was demonstrated with the baseline.

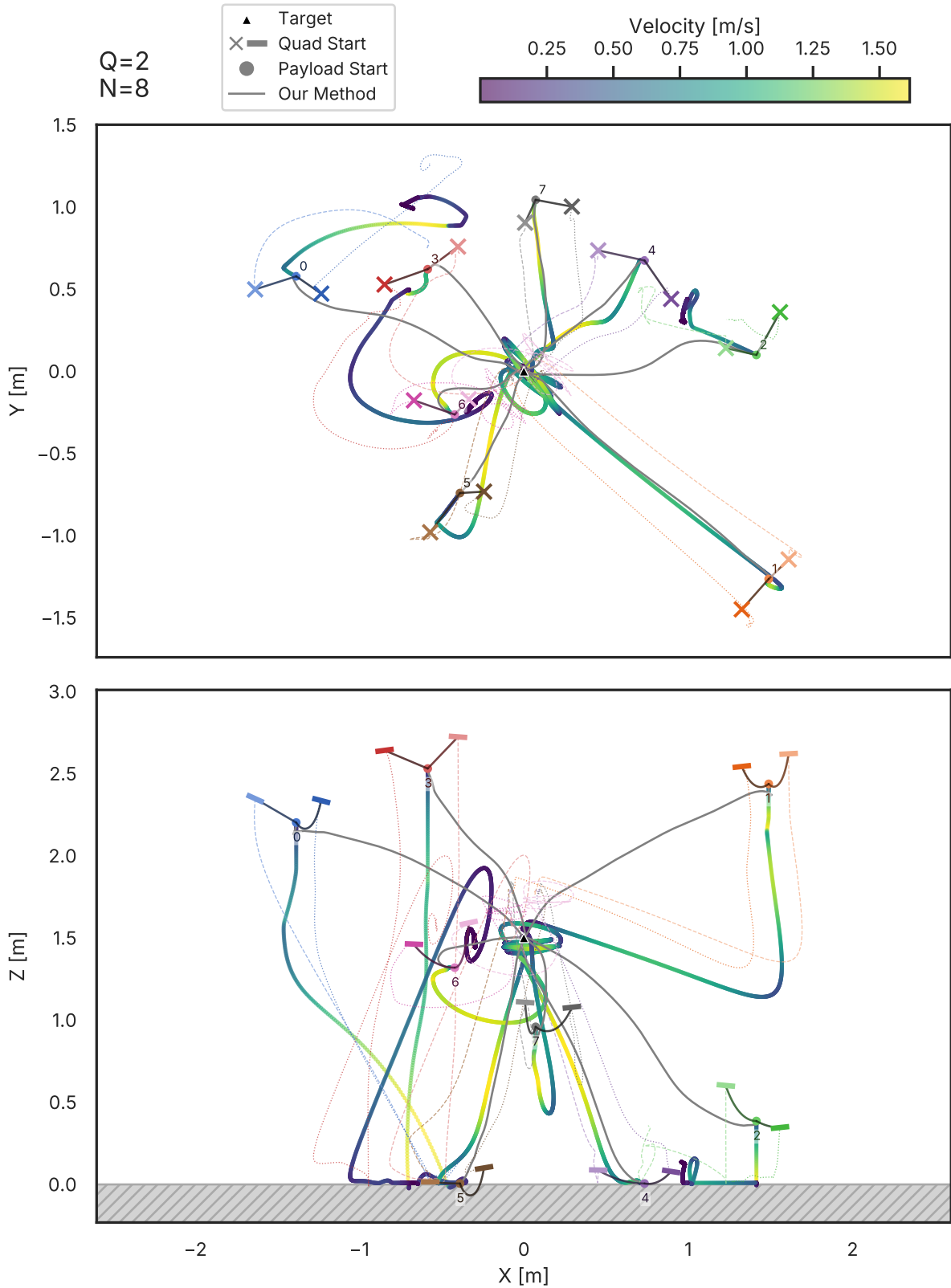


Figure 5.9: Comparison of the learned policy with the baseline method from [13]. Recovery of two quadrotors with payload from harsh conditions. The top plot is a top-down (xy) view, while the bottom one is a side view (xz). The colormapped line shows the baseline method, while the thin gray line shows the learned policy as evaluated before.

5.5 Generalization

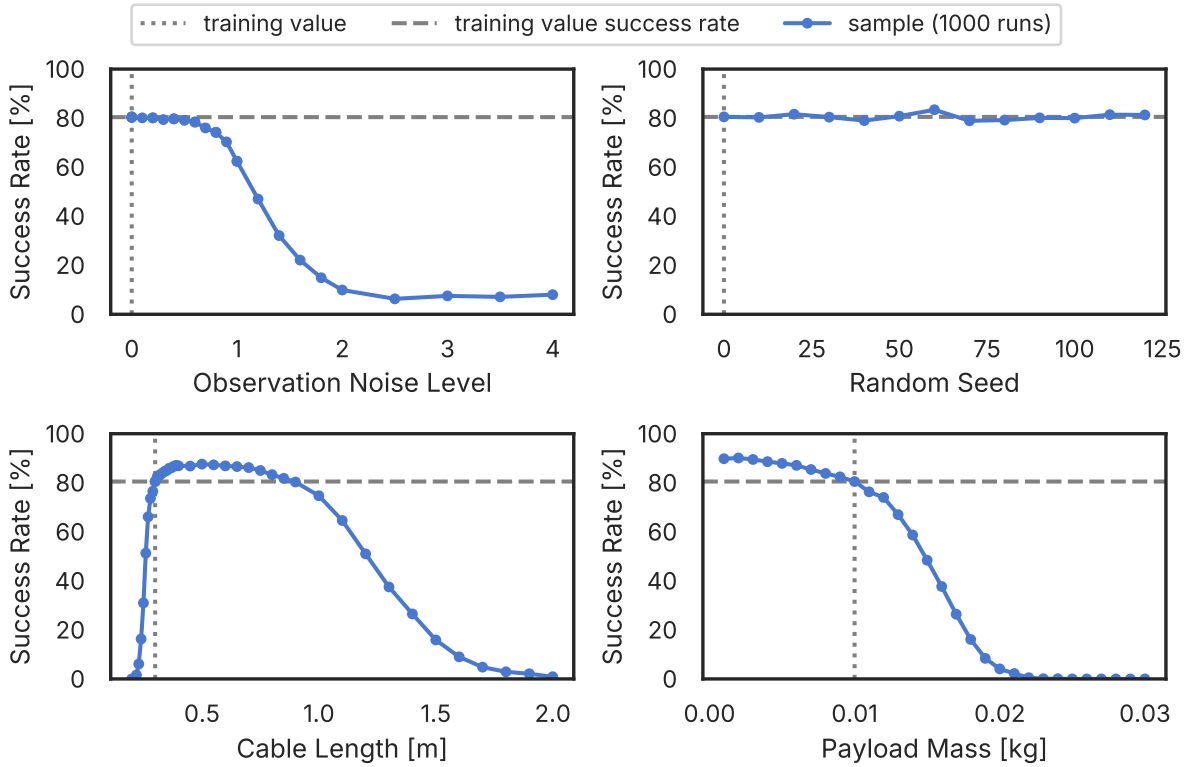


Figure 5.10: Evaluation of the learned policy’s generalization capabilities in the two quadrotors with payload scenario. Each datapoint represents 1000 runs in an environment that only differs in the specific value adjusted. The policy is trained on a 0.3 m cable length and a payload mass of 0.01 kg.

In order to evaluate the generalization capabilities of our learned policies, we conduct experiments in the two quadrotor with payload scenario, where we vary the cable length, payload mass, observation noise level, and seed. For each datapoint, we run the policy for 1000 runs each for 10 s. We only change the one evaluated parameter in the simulation configuration. We evaluate the percentage of runs with successful recoveries in this timeframe.

The results are shown in Figure 5.10. The figure shows that the policy generalizes well to different cable lengths and payload masses. The policy is trained on a 0.1 kg payload mass and a 0.3 m cable length without domain randomization on these parameters. Decreasing the cable length drastically decreases the success rate, while increasing the cable length actually increases the success rate compared to the default configuration. This follows intuition, as quadrotor collisions are way more likely with shorter cables. Only after a cable length of 1 m does the success rate drop again, which is likely due to the payload being too far away from the quadrotors to be stabilized effectively.

The policy also generalizes well to different payload masses, with a slight decrease in performance for lighter payloads. Heavier payloads decrease the success rate.

We also evaluate the generalization to different observation noise scaling levels and seeds. The policy generalizes well to different observation noise levels, with a drastic decrease

in performance for noise levels above 1. The seed generalization shows that the policy generalizes very well across different seeds, thanks to the highly parallelized training on 16k environments.

The generalization capabilities of the policy show that it can adapt to different cable lengths and payload masses, even without domain randomization during training. This is a strong indicator of the robustness of the learned policy and its potential for real-world applications.

5.6 Sim-to-Real Transfer

To transfer the learned policies to real-world quadrotors, we export the trained policies as a TFLite model and transform them with the STM32 Cube AI toolchain to run on the STM32F7 microcontroller on the Crazyflie. We then deploy the policies on a real quadrotor, mapping the internal state estimation of the EKF to the observation space of the policy. The policies are then run on the real quadrotors with a 250 Hz control frequency. We evaluated the policies so far only in the single quadrotor without payload scenario.

So far, we have not been able to achieve a successful sim-to-real transfer. This might still be due to implementation issues or might require additional domain randomization during training. We plan to further investigate this in future work.

5.7 Experimental Results Summary

In this chapter, we evaluated the performance of our learned policies in simulation in both single-quadrotor and multi-quadrotor scenarios. The results demonstrate that our approach effectively stabilizes payloads in harsh conditions and accurately tracks trajectories. The policies generalize well to different initial conditions and quadrotor configurations. We compared our method with the existing baseline and demonstrated its superiority in terms of stability and robustness. We still need to demonstrate sim-to-real transfer, which is a limitation of the current work. Overall, our approach might provide a robust solution for cooperative aerial transport tasks, demonstrating the potential of RL in this domain.

6. Discussion

This chapter brings together the central insights from our work on training and evaluating RL agents for decentralized multi-quadrotor cable-suspended payload transport. We revisit our research objectives and contributions, summarize the most important findings, consider their broader significance, acknowledge the limitations we encountered, and suggest directions for future research to further advance the field.

6.1 Key Findings

In pursuit of our research objectives, we first formulated the decentralized multi UAV payload transport problem as a multi agent Dec-POMDP with local observation and action spaces enabling coordination through decentralized policies, then designed and implemented a high-performance GPU-parallel simulation pipeline to support scalable policy learning, investigated reward design by decomposing the overall objective into tracking and stability components, evaluated the learned policies under harsh initializations to assess resilience relative to classical baselines, and finally demonstrated that our decentralized framework maintains precise tracking and payload stability as the number of UAVs increases.

Concurrently, running training across thousands of parallel simulation environments effectively reduced training time and gradient variance, accelerating convergence to reliable policy performance. Our proposed training pipeline, implemented in JAX and MuJoCo, enables very capable and efficient training.

Randomizing the initial positions of payloads and quadrotors during training was crucial to train a policy to recover from harsh initial conditions. We found that applying domain randomization solely to the maximum thrust parameter, without randomizing parameters such as payload mass, quadrotor mass, and cable length, still leads to decent generalization in simulation. Extending each agent’s observation to include the most recent action further enhanced the stability and smoothness of the control outputs.

The design of the reward function proved to be another key factor. By shaping reward components with exponential decay rather than fixed penalties, we prevented agents from terminating episodes prematurely and enabled them to recover gracefully from disturbances. Our experiments demonstrate that including a regularization term on the agent’s actions is critical to producing smooth control signals and avoiding abrupt bang-bang behavior. This regularization also improves the ability to transfer policies from simulation to real hardware.

Notably, policies trained exclusively on recovery and position holding transferred successfully to dynamic trajectory tracking. This result highlights the flexibility of the learned control strategies. Despite these advances, achieving a reliable sim-to-real transfer remains an open challenge.

6.2 Implications

Our findings underscore the potential of RL for cooperative aerial transport tasks that are difficult to handle with traditional model-based control. The decentralized MARL framework we developed is capable of stabilizing suspended payloads, following precise trajectories, and rejecting external disturbances. Such capabilities are directly relevant to real-world applications, including aerial construction support, infrastructure inspection, and rapid delivery of emergency supplies.

At the same time, our results emphasize that accurate and robust state estimation is essential for any practical deployment. Our approach requires the position information of all quadrotors to be shared with low latency and high frequency. In many real-world environments, full state information is not available, external motion capture systems are impractical, and low-latency communication is challenging. Ensuring that learned policies remain resilient in the face of communication issues, sensor noise, and estimation errors is therefore a critical requirement for future work.

6.3 Limitations

Several important limitations must be mentioned in the conclusions of this study. First, learned policies sometimes fail under extreme conditions, for example, when payload oscillations grow large or quadrotor separation becomes excessive. Our current approach does not include obstacle avoidance or integrated motion planning and relies heavily on external infrastructure without providing formal safety guarantees. Each change in payload mass or quadrotor configuration presently requires retraining from scratch, which limits flexibility. Although we observed some generalization across payload masses and cable lengths in simulation, systematic validation on real platforms is still needed.

Moreover, our methods depend on knowing quantities such as payload mass and cable lengths in advance. Improving autonomy will require onboard estimation of these variables. Addressing the challenge of accurately estimating the full system parameters of cables and payload remains an important barrier to fully autonomous operation.

As mentioned above, our method requires high-frequency position updates from all quadrotors, which is not always feasible in real-world scenarios. The current approach also does not account for communication delays or packet loss, which can significantly impact performance in practice. Future work should explore ways to make the system more robust to these issues, for example, by incorporating predictive control or state estimation techniques, or by estimating the positions of other quadrotors solely from local observations of each quadrotor.

Finally, while our training pipeline is highly efficient, it still requires substantial computational resources for training. Future work should explore more sample-efficient algorithms and methods to reduce the reliance on extensive simulation data.

6.4 Future Research Directions

To address these challenges and enhance the real-world applicability of decentralized RL for multi-quadrotor cable-suspended payload transport, we propose several avenues for future research. Enriching the agent’s state representation with extended histories or estimates of latent variables such as cable length, payload mass, quadrotor mass, and relative positions could yield more robust control. Developing onboard estimation schemes that leverage IMU cameras or LiDAR sensors would remove dependence on external motion capture systems. These should be combined with motion and path planning algorithms to ensure safe operation in dynamic environments. To facilitate real-world deployment, inter-quadrotor communication must be investigated, including methods for handling communication delays and packet loss.

Expanding experiments to a wider range of payload shapes, masses, and cable configurations will help characterize the limits of scalability and generalization. Improving sim-to-real transfer by applying advanced domain randomization methods may reduce the remaining reality gap. Finally, incorporating online policy adaptation mechanisms and exploring alternative RL algorithms, such as off-policy actor-critic methods, recurrent networks, attention mechanisms, or meta-learning approaches, could accelerate training and improve policy flexibility under changing conditions.

Pursuing these directions will bring us closer to realizing fully autonomous, reliable, and efficient multi-agent cable-suspended aerial transport systems based on RL.

7. Conclusion

In this work, we have presented a unified framework for training RL controllers capable of performing both single- and multi-quadrotor cable-suspended payload transport. By leveraging large-scale parallel simulation, carefully designed exponential-decay reward terms, and selective domain randomization of physical parameters, we obtained policies that recover gracefully from severe disturbances and generalize zero-shot from position holding to figure-eight trajectory tracking, outperforming the state-of-the-art baseline methods.

Our experiments demonstrate that regularizing action outputs and including a short history of past commands yields smooth thrust profiles, eliminating bang-bang behavior and enabling preliminary sim-to-real transfer. Cooperative control scales to teams of up to four vehicles, with each additional agent contributing to improved payload stability. However, performance degradation under extreme initial conditions highlights the inherent coordination challenges.

Through statistical analysis of 1,000 recovery rollouts per configuration and qualitative visualization of recovery and trajectory-tracking runs, we have shown that the learned policies achieve a stable hover and follow paths with high accuracy. Domain randomization of thrust coefficients, initial poses, and sensor noise proved sufficient to confer robustness to parameter variations and estimation errors. Holding payload mass and cable length fixed kept learning tractable.

Despite these achievements, several questions remain open. Extending the policies to handle obstacle avoidance, heterogeneous teams, and dynamic environments will require integration of higher-level planning and safety constraints. Moreover, closing the sim-to-real gap in outdoor settings with wind and noise might require extensive domain randomization curricula and online estimation of latent parameters, such as cable tension. Exploring off-policy and meta-learning algorithms may further improve sample efficiency and rapid adaptation to new payloads.

By open-sourcing our software framework **CrazyMarl**, which provides end-to-end high-performance JAX training for Crazyflie swarms, along with our methodology, evaluation scripts, and trained models, we aim to accelerate progress toward robust, fully autonomous aerial transport with a team of quadrotors. The demonstrated ability of RL to produce agile coordinated flight behaviors lays the groundwork for practical applications such as autonomous delivery, infrastructure inspection, and collaborative assembly tasks.

Bibliography

- [1] M. Idrissi, M. Salami, and F. Annaz, “A review of quadrotor unmanned aerial vehicles: Applications, architectural design and control algorithms,” *Journal of Intelligent & Robotic Systems*, vol. 104, pp. 1–33, 2022.
- [2] M. Lyu, Y. Zhao, C. Huang, and H. Huang, “Unmanned aerial vehicles for search and rescue: A survey,” *Remote. Sens.*, vol. 15, p. 3266, 2023.
- [3] J. Chen, J. Sun, and G. Wang, “From unmanned systems to autonomous intelligent systems,” *Engineering*, 2021.
- [4] P. Abbaraju, R. Voyles, et al., “Sensing and sampling of trace contaminations by a dexterous hexrotor uav at nuclear facilities-18600,” in *Proceedings of the WM2018 Symposium Conference, Phoenix, AZ, USA*, 2018, pp. 18–22.
- [5] S. Batra, Z. Huang, A. Petrenko, T. Kumar, A. Molchanov, and G. S. Sukhatme, “Decentralized control of quadrotor swarms with end-to-end deep reinforcement learning,” in *Proceedings of the 5th Conference on Robot Learning*, Jan. 11, 2022, pp. 576–586. DOI: 10.48550/arxiv.2109.07735 eprint: 2109.07735.
- [6] X. Zhou, J. Zhu, H. Zhou, C. Xu, and F. Gao, “Ego-swarm: A fully autonomous and decentralized quadrotor swarm system in cluttered environments,” *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 4101–4107, 2020.
- [7] J. Estevez, G. Garate, J. M. Lopez-Guede, and M. Larrea, “Review of aerial transportation of suspended-cable payloads with quadrotors,” *Drones*, vol. 8, p. 35, Feb. 2024, ISSN: 2504-446X. DOI: 10.3390/drones8020035
- [8] J. Hwangbo, I. Sa, R. Siegwart, and M. Hutter, “Control of a quadrotor with reinforcement learning,” *IEEE Robotics and Automation Letters*, vol. 2, pp. 2096–2103, 2017. DOI: 10.48550/arxiv.1707.05110 eprint: 1707.05110.
- [9] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun, and D. Scaramuzza, “Champion-level drone racing using deep reinforcement learning,” *Nature*, vol. 620, pp. 982–987, Aug. 2023, ISSN: 1476-4687. DOI: 10.1038/s41586-023-06419-4
- [10] Y. Song, A. Romero, M. Müller, V. Koltun, and D. Scaramuzza, “Reaching the limit in autonomous racing: Optimal control versus reinforcement learning,” *Science Robotics*, vol. 8, 2023.
- [11] Z. Huang, Z. Yang, R. Krupani, B. Şenbaşlar, S. Batra, and G. S. Sukhatme, “Collision avoidance and navigation for a quadrotor swarm using end-to-end deep reinforcement learning,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, May 2024, pp. 300–306. DOI: 10.1109/ICRA57147.2024.10611499
- [12] J. Eschmann, D. Albani, and G. Loianno, “Learning to fly in seconds,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6336–6343, Jul. 2024, ISSN: 2377-3766. DOI: 10.1109/LRA.2024.3396025

-
- [13] K. Wahba, J. Ortiz-Haro, M. Toussaint, and W. Hönig, “Kinodynamic motion planning for a team of multirotors transporting a cable-suspended payload in cluttered environments,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct. 2024, pp. 12 750–12 757. DOI: 10.1109/IR0S58592.2024.10802794
 - [14] J. Chen et al., “What matters in learning a zero-shot sim-to-real rl policy for quadrotor control? a comprehensive study,” *IEEE Robotics and Automation Letters*, vol. 10, no. 7, pp. 7134–7141, Jul. 2025, ISSN: 2377-3774. DOI: 10.1109/lra.2025.3575011
 - [15] M. L. Littman, “Markov games as a framework for multi-agent reinforcement learning,” in *Machine learning proceedings 1994*, 1994, pp. 157–163.
 - [16] K. Sreenath and V. Kumar, “Dynamics, control and planning for cooperative manipulation of payloads suspended by cables from multiple quadrotor robots,” in *Robotics: Science and Systems IX*, Jun. 23, 2013, ISBN: 978-981-07-3937-9. DOI: 10.15607/RSS.2013.IX.011
 - [17] D. K. D. Villa, A. S. Brandão, R. Carelli, and M. Sarcinelli-Filho, “Cooperative load transportation with two quadrotors using adaptive control,” *IEEE Access*, vol. 9, pp. 129 148–129 160, 2021, ISSN: 2169-3536. DOI: 10.1109/ACCESS.2021.3113466
 - [18] T. Lee, “Geometric control of quadrotor UAVs transporting a cable-suspended rigid body,” *IEEE Transactions on Control Systems Technology*, vol. 26, pp. 255–264, Jan. 2018, ISSN: 1558-0865. DOI: 10.1109/TCST.2017.2656060
 - [19] M. Tognon, C. Gabellieri, L. Pallottino, and A. Franchi, “Aerial co-manipulation with cables: The role of internal force for equilibria, stability, and passivity,” *IEEE Robotics and Automation Letters*, vol. 3, pp. 2577–2583, Jul. 2018, ISSN: 2377-3766. DOI: 10.1109/LRA.2018.2803811
 - [20] A. Tagliabue, M. Kamel, R. Siegwart, and J. I. Nieto, “Robust collaborative object transportation using multiple mavs,” *Int. J. Robotics Res.*, vol. 38, no. 9, 2019. DOI: 10.1177/0278364919854131
 - [21] S. Sun and A. Franchi, “Nonlinear MPC for full-pose manipulation of a cable-suspended load using multiple UAVs,” in *2023 International Conference on Unmanned Aircraft Systems (ICUAS)*, Jun. 2023, pp. 969–975. DOI: 10.1109/ICUAS57906.2023.10156031
 - [22] G. Li and G. Loianno, “Nonlinear model predictive control for cooperative transportation and manipulation of cable suspended payloads with multiple quadrotors,” in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct. 2023, pp. 5034–5041. DOI: 10.1109/IR0S55552.2023.10341785 eprint: 2303.06165.
 - [23] B. E. Jackson, T. A. Howell, K. Shah, M. Schwager, and Z. Manchester, “Scalable cooperative transport of cable-suspended loads with UAVs using distributed trajectory optimization,” *IEEE Robotics and Automation Letters*, vol. 5, pp. 3368–3374, Apr. 2020, ISSN: 2377-3766. DOI: 10.1109/LRA.2020.2975956
 - [24] C. Gabellieri, M. Tognon, D. Sanalidro, and A. Franchi, “Equilibria, stability, and sensitivity for the aerial suspended beam robotic system subject to parameter uncertainty,” *IEEE Transactions on Robotics*, vol. 39, pp. 3977–3993, Oct. 2023, ISSN: 1941-0468. DOI: 10.1109/TR0.2023.3279033 eprint: 2302.07031.
 - [25] Y. Wang, J. Wang, X. Zhou, T. Yang, C. Xu, and F. Gao, “Safe and agile transportation of cable-suspended payload via multiple aerial robots,” *CoRR*, vol. abs/2501.15272, 2025. DOI: 10.48550/ARXIV.2501.15272 arXiv: 2501.15272.
-

-
- [26] W. Koch, R. Mancuso, R. West, and A. Bestavros, “Reinforcement learning for uav attitude control,” *ACM Transactions on Cyber-Physical Systems*, vol. 3, pp. 1–21, 2018. DOI: 10.48550/arxiv.1804.04154 eprint: 1804.04154.
 - [27] T. P. Lillicrap et al., “Continuous control with deep reinforcement learning,” *CoRR*, vol. abs/1509.02971, 2015. DOI: 10.48550/arxiv.1509.02971 eprint: 1509.02971.
 - [28] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, “Trust region policy optimization,” *CoRR*, vol. abs/1502.05477, 2015. DOI: 10.48550/arxiv.1502.05477 arXiv: 1502.05477. [Online]. Available: <http://arxiv.org/abs/1502.05477>
 - [29] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *CoRR*, vol. abs/1707.06347, 2017. DOI: 10.48550/arxiv.1707.06347 arXiv: 1707.06347. [Online]. Available: <http://arxiv.org/abs/1707.06347>
 - [30] J. Xing, I. Geles, Y. Song, E. Aljalbout, and D. Scaramuzza, “Multi-task reinforcement learning for quadrotors,” *IEEE Robotics and Automation Letters*, pp. 1–8, 2024, ISSN: 2377-3766. DOI: 10.1109/LRA.2024.3520894
 - [31] A. Molchanov, T. Chen, W. Hönig, J. A. Preiss, N. Ayanian, and G. S. Sukhatme, “Sim-to-(multi)-real: Transfer of low-level robust control policies to multiple quadrotors,” in *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Nov. 2019, pp. 59–66. DOI: 10.1109/IROS40897.2019.8967695
 - [32] E. Kaufmann, L. Bauersfeld, and D. Scaramuzza, “A benchmark comparison of learned control policies for agile quadrotor flight,” in *2022 International Conference on Robotics and Automation (ICRA)*, May 2022, pp. 10 504–10 510. DOI: 10.1109/ICRA46639.2022.9811564
 - [33] S. Gronauer, M. Kissel, L. Sacchetto, M. Korte, and K. Diepold, “Using simulation optimization to improve zero-shot policy transfer of quadrotors,” in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, Oct. 2022, pp. 10 170–10 176. DOI: 10.1109/IROS47612.2022.9981229 eprint: 2201.01369.
 - [34] H. Ma, Z. Ren, B. Dai, and N. Li, “Skill transfer and discovery for sim-to-real learning: A representation-based viewpoint,” *technical report*, 2024. DOI: 10.48550/arxiv.2404.05051 eprint: 2404.05051.
 - [35] Z. Huang et al., “Quadswarm: A modular multi-quadrotor simulator for deep reinforcement learning with direct thrust control,” *CoRR*, vol. abs/2306.09537, 2023. DOI: 10.48550/ARXIV.2306.09537 arXiv: 2306.09537.
 - [36] S. Belkhale, R. Li, G. Kahn, R. McAllister, R. Calandra, and S. Levine, “Model-based meta-reinforcement learning for flight with suspended payloads,” *IEEE Robotics and Automation Letters*, vol. 6, pp. 1471–1478, Apr. 2021, ISSN: 2377-3766, 2377-3774. DOI: 10.1109/LRA.2021.3057046 eprint: 2004.11345.
 - [37] H. Hua, Y. Fang, X. Zhang, and C. Qian, “A new nonlinear control strategy embedded with reinforcement learning for a multirotor transporting a suspended payload,” *IEEE/ASME Transactions on Mechatronics*, vol. 27, pp. 1174–1184, Apr. 2022, ISSN: 1941-014X. DOI: 10.1109/TMECH.2021.3082897
 - [38] C. A. Dimmig et al., “Survey of simulators for aerial robots: An overview and in-depth systematic comparisons [survey],” *IEEE Robotics Autom. Mag.*, vol. 32, no. 2, pp. 153–166, 2025. DOI: 10.1109/MRA.2024.3433171 arXiv: 2311.02296.
 - [39] G. Li, X. Liu, and G. Loianno, “Rotortm: A flexible simulator for aerial transportation and manipulation,” *IEEE Transactions on Robotics*, vol. 40, pp. 831–850, 2022.
-

-
- [40] M. Kulkarni, W. Rehberg, and K. Alexis, "Aerial gym simulator: A framework for highly parallelized simulation of aerial robots," *IEEE Robotics and Automation Letters*, vol. 10, pp. 4093–4100, 2025. DOI: 10.1109/LRA.2025.3548507
 - [41] B. Xu, F. Gao, C. Yu, R. Zhang, Y. Wu, and Y. Wang, "OmniDrones: An efficient and flexible platform for reinforcement learning in drone control," *IEEE Robotics and Automation Letters*, vol. 9, pp. 2838–2844, Mar. 2024, ISSN: 2377-3766. DOI: 10.1109/LRA.2024.3356168
 - [42] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," *CoRR*, vol. abs/1706.02275, 2017. DOI: 10.48550/arxiv.1706.02275 arXiv: 1706.02275. [Online]. Available: <http://arxiv.org/abs/1706.02275>
 - [43] C. Yu et al., "The surprising effectiveness of PPO in cooperative multi-agent games," in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., 2022. [Online]. Available: http://papers.nips.cc/paper_files/paper/2022/hash/9c1535a02f0ce079433344e14d910597-Abstract-Datasets_and_Benchmarks.html
 - [44] C. S. de Witt et al., "Is independent learning all you need in the starcraft multi-agent challenge?" *CoRR*, vol. abs/2011.09533, 2020. DOI: 10.48550/arxiv.2011.09533 arXiv: 2011.09533. [Online]. Available: <https://arxiv.org/abs/2011.09533>
 - [45] G. Papoudakis, F. Christianos, L. Schäfer, and S. V. Albrecht, "Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks," in *NeurIPS Datasets and Benchmarks*, 2020.
 - [46] Y. Chen et al., "Towards human-level bimanual dexterous manipulation with reinforcement learning," *CoRR*, vol. abs/2206.08686, 2022. DOI: 10.48550/ARXIV.2206.08686 arXiv: 2206.08686.
 - [47] B. Pandit et al., "Learning decentralized multi-biped control for payload transport," in *Conference on Robot Learning*, 2024. DOI: 10.48550/arxiv.2406.17279 eprint: 2406.17279.
 - [48] Y. Ji, B. Zhang, and K. Sreenath, "Reinforcement learning for collaborative quadrupedal manipulation of a payload over challenging terrain," *2021 IEEE 17th International Conference on Automation Science and Engineering (CASE)*, pp. 899–904, 2021. DOI: 10.1109/case49439.2021.9551481
 - [49] W. Chen, M. Nguyen, Z. Li, G. N. Sue, and K. Sreenath, "Decentralized navigation of a cable-towed load using quadrupedal robot team via MARL," *CoRR*, vol. abs/2503.18221, 2025. DOI: 10.48550/ARXIV.2503.18221 arXiv: 2503.18221.
 - [50] Y. Xie et al., "Multi-uav behavior-based formation with static and dynamic obstacles avoidance via reinforcement learning," *CoRR*, vol. abs/2410.18495, 2024. DOI: 10.48550/ARXIV.2410.18495 arXiv: 2410.18495.
 - [51] X. Diao and J. Wang, "Efficient multi-agent navigation with lightweight DRL policy," *CoRR*, vol. abs/2408.16370, 2024. DOI: 10.48550/ARXIV.2408.16370 arXiv: 2408.16370.
 - [52] Z. Zhao, Y. Wan, and Y. Chen, "Deep reinforcement learning-driven collaborative rounding-up for multiple unmanned aerial vehicles in obstacle environments," *Drones*, vol. 8, p. 464, Sep. 2024, ISSN: 2504-446X. DOI: 10.3390/drones8090464
 - [53] D. Lin, J. Han, K. Li, J. Zhang, and C.-y. Zhang, "Payload transporting with two quadrotors by centralized reinforcement learning method," *IEEE Transactions on*
-

- Aerospace and Electronic Systems*, vol. 60, pp. 239–251, 2024. DOI: 10.1109/taes.2023.3321260
- [54] J. Estevez, J. Manuel Lopez-Guede, J. del Valle-Echavarri, and M. Graña, “Reinforcement learning based trajectory planning for multi-uav load transportation,” *IEEE Access*, vol. 12, pp. 144 009–144 016, 2024. DOI: 10.1109/ACCESS.2024.3470509
 - [55] V. Mnih et al., “Human-level control through deep reinforcement learning,” *nature*, vol. 518, pp. 529–533, 2015.
 - [56] R. S. Sutton and A. G. Barto, *Reinforcement learning - an introduction*, 2nd Edition. MIT Press, 2018. [Online]. Available: <http://www.incompleteideas.net/book/the-book-2nd.html>
 - [57] F. A. Oliehoek and C. Amato, *A Concise Introduction to Decentralized POMDPs* (Springer Briefs in Intelligent Systems). Cham: Springer, 2016, ISBN: 978-3-319-28927-4. DOI: 10.1007/978-3-319-28929-8
 - [58] F. A. Goodarzi and T. Lee, “Dynamics and control of quadrotor UAVs transporting a rigid body connected via flexible cables,” in *2015 American Control Conference (ACC)*, Jul. 2015, pp. 4677–4682. DOI: 10.1109/ACC.2015.7172066
 - [59] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *2012 IEEE/RSJ international conference on intelligent robots and systems*, Oct. 2012, pp. 5026–5033. DOI: 10.1109/IRoS.2012.6386109
 - [60] A. Rutherford et al., “Jaxmarl: Multi-agent RL environments and algorithms in JAX,” in *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, A. Globersons et al., Eds., 2024. [Online]. Available: http://papers.nips.cc/paper_files/paper/2024/hash/5aee125f052c90e326dcf6f380df94f6-Abstract-Datasets_and_Benchmarks_Track.html
 - [61] R. T. Lange, *gymnax: A JAX-based reinforcement learning environment library*, version 0.0.4, 2022. [Online]. Available: <http://github.com/RobertTLange/gymnax>